

TRAVAUX DE FIN DE CYCLE

En vue de l'obtention du diplôme de licence professionnelle en informatique
Option : Génie logiciel

Thème :
**DEVELOPPEMENT D'UNE APPLICATION DE
MISE A DISPOSITION DES OUVRIERS**

Présentés et soutenus publiquement par :

- **NGATSE Beniche Thechany gloire**
- **KUBEMBULA Jean Elie**

Dirigés par :

Monsieur **KITOKO Roland Alexis**
Ingénieur informaticien et télécoms,
chargé de cours au CFI-CRAS

Promotion 2022

Dédicace

Nous avons l'honneur de dédier ce projet de fin d'études à nos familles qui nous ont soutenus sans relâche tout au long de ce parcours universitaire. À nos parents, qui ont cru en nous dès le début et qui nous ont apporté leur amour inconditionnel, leur encouragement et leur soutien financier. A nos frères et sœurs, qui ont été là pour nous dans les moments difficiles et qui ont partagé notre enthousiasme dans les moments de réussite.

Plus précisément à :

- KOUKAKANA PHILIPPINE
- NGATSE MARDOCHEE
- NGATSE PHANELLE
- NGATSE BROCHELLA
- NGATSE MAURICE

NGATSE BENICHE THECHANY GLOIRE

Aussi à :

- MADEDE MASSAKA SYLVIE
- SERGE AIME DZABATOU ECKO
- JEAN NOËL MAVOUNGOU

KUBEMBULA JEAN ELIE

Nous dédions également ce travail à nos professeurs et mentors qui ont nourri notre passion pour ce domaine et nous ont transmis leur expertise avec patient et diligence. Votre guidance et vos conseils nous ont été précieux tout au long de ce processus et nous ne pourrions pas être là où nous en sommes aujourd'hui sans votre soutien.

Enfin, nous dédions ce projet de fin d'étude à tous ceux qui ont contribué de près ou de loin à notre formation académique.

Nous espérons que ce travail reflète notre reconnaissance pour toutes les personnes qui ont joué un rôle dans notre parcours académique et qui nous ont aidés à atteindre cet objectif.

Remerciements

Au Docteur **Alain EKONDZI**, **Directeur** du **Centre de Formation en Informatique du CIRAS**. Nous avons l'honneur de vous adresser nos plus sincères remerciements pour votre soutien à notre formation professionnelle supérieure. Votre engagement pour l'excellence académique et votre détermination à nous fournir les meilleures ressources pour notre formation ont été d'un grand apport éducatif pour nous.

A monsieur **Roland KITOKO**, nous tenons à vous exprimer notre profonde gratitude pour votre soutien et votre encadrement tout au long de ce projet de projet de fin d'étude. Nous sommes reconnaissants pour le temps que vous avez consacré à notre formation.

Nous remercions les membres du Jury pour le temps qu'ils consacreront à l'évaluation de notre projet de fin d'étude.

Un grand merci à nos parents pour leur soutien tant moral que matériel tout au long de notre formation.

Pour finir, nous n'oublions pas nos condisciples du CFI-CIRAS avec qui nous avons partagé notre expérience d'apprentissage tout au long de notre parcours académique. Nous les remercions chaleureusement pour tous leur soutien et aide précieuse.

Liste des figures

FIGURE 1 : LOGO UML	4
FIGURE 2 : PROCESSUS UNIFIE (CYCLE DE VIE)	7
FIGURE 3 : PROCESSUS 2TUP	8
FIGURE 4 : DEMANDE DE DEVIS MON-OUVRIER	11
FIGURE 5 : SERVICES PROPOSES MON-OUVRIER	11
FIGURE 6 : DEMANDE DE DEVIS HABITATPRESTO	12
FIGURE 7 : CREATION D'UN COMPTE	13
FIGURE 8 : CREATION D'UN COMPTE PROFESSIONNEL HABITATPRESTO	13
FIGURE 9 : DIAGRAMME DE CONTEXTE STATIQUE	17
FIGURE 10 : AJOUTER UN DOMAINE	24
FIGURE 11 : CREER UN COMPTE	25
FIGURE 12 : DIAGRAMME DE SEQUENCE (AJOUTER UN DOMAINE)	26
FIGURE 13 : DIAGRAMME DE SEQUENCE (CREER UN COMPTE)	27
FIGURE 14 : DIAGRAMME D'ACTIVITE AJOUTER UN DOMAINE	28
FIGURE 15 : DIAGRAMME D'ACTIVITE CREER UN COMPTE	29
FIGURE 16 : DIAGRAMME DE CAS D'UTILISATION (REFERENTIEL DES DOMAINES BTP)	30
FIGURE 17 : DIAGRAMME DE CAS D'UTILISATION (GESTION DES OFFRES ET RESERVATION)	31
FIGURE 18 : DIAGRAMME DE CAS D'UTILISATION (GESTION DES QUALIFICATIONS DES OUVRIERS)	31
FIGURE 19 : DIAGRAMME DE CAS D'UTILISATION (GESTION DES DISCUSSIONS)	32
FIGURE 20 : DIAGRAMME DE CAS D'UTILISATION (GESTION DES COMPTES UTILISATEUR)	32
FIGURE 21 : DIAGRAMME DE CLASSE DU REFERENTIEL DU DOMAINE BTP	34
FIGURE 22 : DIAGRAMME DE CLASSE (GESTION DES OFFRES ET RESERVATIONS)	34
FIGURE 23 : DIAGRAMME DE CLASSE (GESTION DES QUALIFICATIONS DES OUVRIERS)	35
FIGURE 24 : DIAGRAMME DE CLASSE (GESTION DES COMPTES UTILISATEURS)	35
FIGURE 25 : DIAGRAMME DE CLASSE GLOBAL DU SYSTEME	36
FIGURE 26 : ARCHITECTURE LOGICIELLE	39
FIGURE 27 : DIAGRAMME DE COMPOSANT	39
FIGURE 28 : DIAGRAMME DE DEPLOIEMENT	45
FIGURE 29 : SCHEMA RELATIONNEL	46
FIGURE 30 : MODELE PHYSIQUE	48

Liste des tableaux

<i>TABLEAU 1 : ABREVIATION ET SIGLES</i>	<i>V</i>
<i>TABLEAU 2 : TABLEAU COMPARATIF</i>	<i>14</i>
<i>TABLEAU 3 : CLASSIFICATION PAR PACKAGE</i>	<i>20</i>
<i>TABLEAU 4 : CLASSIFICATION DES CAS D'UTILISATION (RISQUE, PRIORITE, ITERATION)</i>	<i>23</i>
<i>TABLEAU 5 : ENVIRONNEMENT MATERIEL</i>	<i>44</i>

Abréviations & sigles

Sigles	Significations
API	Application Programming Interface (Interface de programmation d'applications)
AVD	Android Virtual Machine (Machine virtuelle Android)
BTP	Bâtiment travaux publics
CFI	Centre de Formation en Informatique
CIRAS	Centre Informatique de Recherche de l'Armée et de la sécurité
HTTP	Hyper Text Transfer Protocol (Protocole de transfert hypertexte)
IDE	Integrated Development Environment (Environnement de développement intégré)
JSON	JavaScript Object Notation (Notation d'objet JavaScript)
MLD	Modèle Logique de Données
MPD	Modèle Physique de Données
MVC	Model View Controller (Modèle Vue Contrôleur)
NTIC	Nouvelles Technologies de l'information et de la communication
OS	Operating system (Système d'exploitation)
PC	Personal Computer (Ordinateur personnel)
SDK	Software Development Kits (Kits de développement logiciel)
SGBD	Système de Gestion de Base de Données
SQL	Structured Query Language (Langage de requêtes structurées)
UML	Unified Modeling Language (Langage de modélisation unifié)
UP	Unified Process (Processus unifié)
2TUP	Two Tracks Unified Process (Processus unifié à deux branches)

Tableau 1 : Abréviation et sigles

Sommaire

REMERCIEMENTS	II
LISTE DES FIGURES	III
LISTE DES TABLEAUX	IV
ABREVIATIONS & SIGLES	V
SOMMAIRE	VI
INTRODUCTION	1
CHAPITRE 1 : PRESENTATION DE LA PROBLEMATIQUE	2
CHAPITRE 2 : APPROCHE METHODOLOGIQUE	4
CHAPITRE 3 : ETUDE DE L'EXISTANT	10
CHAPITRE 4 : ANALYSE	16
CHAPITRE 5 : CONCEPTION.....	30
CHAPITRE 6 : IMPLEMENTATION	40
CONCLUSION.....	56
BIBLIOGRAPHIE & WEBOGRAPHIE.....	A

Introduction

Les plateformes de mise à disposition d'ouvriers dans le domaine du BTP sont devenues de plus en plus populaire ces dernières années, en particulier dans ce secteur. Nous avons voulu explorer les différents aspects de ce phénomène pour comprendre sa pertinence et son impact sur l'industrie du BTP.

L'analyse du secteur révèle que les plateformes en ligne dédiées à la mise à disposition d'ouvriers ont connu une croissance significative au cours des dernières années. Ces plateformes offrent aux employeurs la possibilité de trouver facilement et rapidement des ouvriers qualifiés pour accomplir une diversité des travaux, apportant ainsi une solution souple aux besoins en main-d'œuvre de ce secteur. Les ouvriers ont la latitude de choisir les travaux qu'ils souhaitent exécuter ainsi que leurs horaires de travail, tandis que les employeurs ont la capacité de sélectionner les ouvriers les mieux adaptés à leurs exigences spécifiques.

Aujourd'hui, de nombreux ouvriers du BTP choisissent de travailler en freelance plutôt que d'être employés par une entreprise, ce qui les aide à maintenir une certaine flexibilité et à mieux concilier leur vie professionnelle et personnelle. Les employeurs, quant à eux peuvent accéder rapidement à des ouvriers qualifiés pour des projets à court terme sans avoir à embaucher du personnel permanent.

Dans le cas du Congo Brazzaville, où les plateformes de mise à disposition des ouvriers en ligne sont quasiment inexistantes, les employeurs rencontrent généralement des difficultés pour trouver rapidement des ouvriers qualifiés. Dans la plupart des cas, les ouvriers sont recrutés par le biais de recommandations personnelles ou par le biais de contacts.

Ainsi pour résoudre ce problème, nous avons choisi, dans le cadre de nos travaux académique, d'étudier et concevoir une plateforme numérique de mise à disposition des ouvriers en ligne, nous examinerons les différents avantages qu'elle apporterait aux ouvriers et employeurs congolais.

Afin de conduire ces travaux, nous avons choisi d'utiliser la méthode UML + 2TUP (Two Tracks Unified Process, en français Processus unifié à deux branches) pour concevoir et mettre en place cette plateforme.

Chapitre 1 : Présentation de la problématique

1.1 Domaine

Notre projet découle de la sélection d'un projet parmi ceux retenus au cours de l'année académique par la coordination des cycles du CFI-CIRAS. Le thème choisi pour notre projet est le suivant : << Développement d'une application de mise à disposition des ouvriers.>>

1.2 Contexte du domaine

Le secteur du BTP englobe un large éventail lié à la conception, à la construction et à la rénovation d'une variété de structures, qu'elles soient publiques ou privées, industrielles ou non. De la phase de construction à celle de la démolition, en passant par l'aménagement et l'entretien, le secteur du Bâtiment et des Travaux Publics (BTP) présente une grande diversité d'activités. En raison de sa nature locale non délocalisable, ce secteur compte de nombreuses entreprises et employés, en faisant l'un des domaines les plus importants en termes de tailles d'entreprises et de main-d'œuvre.

1.3 Problématique du sujet

Au Congo, l'urbanisation rapide en cours devrait s'intensifier dans les prochaines décennies. La nécessité d'ajuster l'offre et la demande de services dans le secteur du BTP est en croissance constante depuis plusieurs années. Cependant, l'absence de plateformes en ligne pour la mise à disposition des ouvriers engendre des lacunes entravant cet équilibre entre l'offre et la demande de services dans le secteur du BTP.

1.4 Intérêt du sujet

Ce projet vise à améliorer le processus d'ajustement entre l'offre et la demande d'activités du BTP au Congo en créant une plateforme numérique. Cette plateforme permettra de mettre à disposition des employeurs des ouvriers qualifiés répondant au mieux à leurs besoins.

1.5 Concept lié au sujet

- **Domaine du BTP** : C'est le regroupement de toutes activités de conceptions et de constructions des bâtiments publics et privés, industrie ou non, et des infrastructures telles que les routes ou les canalisations [1].
- **Particulier** : est une personne qui a besoin de services.
- **Application mobile** : est un logiciel conçu pour être utilisé sur des appareils mobiles tels que des smartphones et des tablettes.
- **Framework de développement** : Ensemble de bibliothèques et d'outils conçus pour aider les développeurs à créer des applications plus rapidement et plus efficacement.

- **Architecture logicielle :** l'architecture est la façon dont les composants logiciels sont organisés et intégrés les uns aux autres pour former une application fonctionnelle. Elle décrit les différents éléments du logiciel, leur rôle et comment ils interagissent entre eux. Il définit également les normes et les conventions de programmation qui sont utilisées pour construire le logiciel.

Chapitre 2 : Approche Méthodologique

Dans ce chapitre, nous présentons l'approche méthodologique adoptée. Nous allons examiner les langages de modélisation ainsi que le processus de développement de notre projet informatique.

2.1 Langage de modélisation

Un langage de modélisation est un type de langage formel utilisé pour représenter des concepts ou des systèmes complexes dans un format structuré et compréhensible [2].

2.2 Présentation d'UML



Figure 1 : Logo UML

UML (Unified Modeling Language) est un langage de modélisation graphique pour représenter les concepts et les systèmes complexes en informatique et dans d'autres domaines. Il a été standardisé par l'Object Management Group (OMG) en 1997 et est devenu un des langages de modélisation les couramment utilisés dans le monde.

UML peut être utilisé pour représenter une variété de concepts et de systèmes, tels que les systèmes d'informations, les applications, les processus métiers, les réseaux. Il permet aux développeurs de visualiser les relations entre les différents éléments d'un système, ce qui facilite la planification, la conception et la mise en œuvre du système.

UML comprend une série de diagrammes, chacun étant spécialement conçu pour représenter un aspect différent d'un système.

UML est divisé en plusieurs sous-ensembles, chacun se concentrant sur un aspect spécifique de la modélisation du système.

- **Les vues** sont les observables du système, elles fournissent une description du système d'un point de vue organisationnel, dynamique, temporel, architectural, géographique, logique, ... La description complète d'un système peut être obtenue

par une combinaison de toutes les vues (logiques, d'implémentation, de déploiement, des processus et des cas d'utilisation).

- **Les diagrammes** sont des représentations graphiques d'une ou plusieurs vues du système. Ils utilisent des symboles et des notations spécifiques à UML pour représenter les différents éléments du système, tels que les classes, les objets, les activités, les interactions.
- **Les modèles d'éléments** sont des spécifications détaillées des différents types d'éléments qu'on inclut dans les diagrammes UML.

2.3 Les types de diagrammes d'UML

La dernière version d'UML, version 2.5.1, qui comprend quatorze diagrammes, à savoir sept diagrammes structurels et sept diagrammes comportementaux. Cependant, UML n'est pas une méthode rigide et l'utilisation de ces diagrammes est laissée à la discrétion de chaque utilisateur. Ces diagrammes sont repartis en trois (3) groupes : les diagrammes structurels ou statiques, les diagrammes comportementaux et les diagrammes d'interactions ou dynamiques.

2.3.1.1 Les diagrammes structurels ou statiques

Les diagrammes structurels ou statiques comprennent les diagrammes qui montrent la structure statique d'un système :

- **Le diagramme de classes** : représente les classes, les objets et les relations entre eux.
- **Le diagramme d'objets** : il permet de visualiser les interactions entre les objets.
- **Le diagramme de déploiement** : montre la topologie physique d'un système, en indiquant où sont déployés les différents composants.
- **Le diagramme de composants** : représente les composants du système et les relations entre eux, en se concentrant sur la structure du système.
- **Le diagramme de paquetages** : Il représente les dépendances entre les paquetages.
- **Le diagramme de structure composite** : C'est la description des relations entre les composants d'une classe.

2.3.1.2 Les diagrammes comportementaux

Les diagrammes comportementaux comprennent les diagrammes qui montrent les interactions dynamiques entre les objets d'un système :

- **Le diagramme de cas d'utilisation** modélise les fonctionnalités et les interactions entre les utilisateurs et le système.
- **Le diagramme d'états-transitions** modélise les différents états d'un objet et les transitions entre ces états.
- **Le diagramme d'activité** modélise la topologie physique d'un système, en indiquant où sont déployés les différents composants.

2.3.1.3 Les diagrammes d'interaction ou dynamique

Il fusionne les diagrammes d'activités et de séquence pour combiner des fragments d'interaction avec des décisions et des flots.

- **Le diagramme de séquence** représente les interactions entre les objets dans un ordre temporel, en montrant les messages échangés entre les objets.
- **Le diagramme de communication** modélise la communication entre objets dans le plan au sein d'une interaction.
- **Le diagramme global d'interactions** est une description des enchaînements possibles entre les scénarii préalablement identifiés sous forme de diagrammes de séquences.
- **Le diagramme de temps** sert à la description des variations d'une donnée au cours du temps.

2.4 Processus de développement

Un processus de développement logiciel est un ensemble organisé d'étapes qui contribuent à la création ou à l'amélioration d'un logiciel. Il est divisé en deux axes : la gestion du développement technique de la solution et la gestion du projet de développement de la solution technique. Le développement se concentre sur la production du logiciel, tandis que la gestion du développement mesure et planifie les coûts et les délais.

2.5 Présentation du Processus Unifié (UP)

Le processus unifié (Unified Process) est un processus de développement logiciel, itératif et incrémental, centré sur l'architecture, piloté par les cas d'utilisation.

Avec le processus unifié, le développement d'un produit logiciel ne se fait pas d'un coup. Le projet est subdivisé en de mini projets qui constituent des itérations. Chaque itération comprend les différentes étapes de l'enchaînement d'activités (expression des besoins, analyse, conception, implémentation et test). L'achèvement d'une itération donne un incrément.

L'architecture définit le contexte de travail de chaque itération. Les cas d'utilisation quant à eux permettent d'établir les objectifs et d'orienter le travail de chaque itération. L'architecture d'un système est une description du système sous différentes vues. Le processus unifié produit une représentation du système sous les vues décrites par UML (la vue des cas d'utilisation, la vue logique, la vue d'implémentation, la vue des processus et la vue de déploiement). Cette représentation constitue l'architecture système.

Le processus unifié répète un certain nombre de fois une série de cycles. Chaque cycle se termine par la livraison d'une version du logiciel. Un cycle comprend quatre (4) phases qui sont la création, l'élaboration, la construction et la traduction. Chacune de ses phases est constituée d'itérations.

La phase de création est la première étape du projet. Elle vise à déterminer le but et les objectifs du projet, à identifier les exigences fonctionnelles et non fonctionnelles et à établir un plan pour le développement du logiciel.

La phase d'élaboration a pour but de développer une vision plus claire du projet et de sa direction, en détaillant les exigences et en élaborant un plan de développement détaillé. Les activités clés comprennent la création d'un plan de test et la définition des architectures logicielles.

La phase de construction consiste à développer le logiciel en utilisant les informations obtenues lors des phases précédentes. Les développeurs peuvent utiliser des méthodes telles que l'analyse itérative, la programmation orientée objet et la programmation incrémentale pour développer le logiciel.

La phase de transition a pour but est de tester le logiciel développé, de le mettre en production et de le livrer aux utilisateurs finaux. Les activités clés comprennent les tests du logiciel, la formation des utilisateurs et la mise en place de la documentation.

Le processus unifié est un processus de développement complet. Il apporte une structure pour le développement d'un produit répondant aux exigences et attentes des utilisateurs. Sa segmentation en cycles permet une surveillance et une gestion plus rigoureuse du développement.

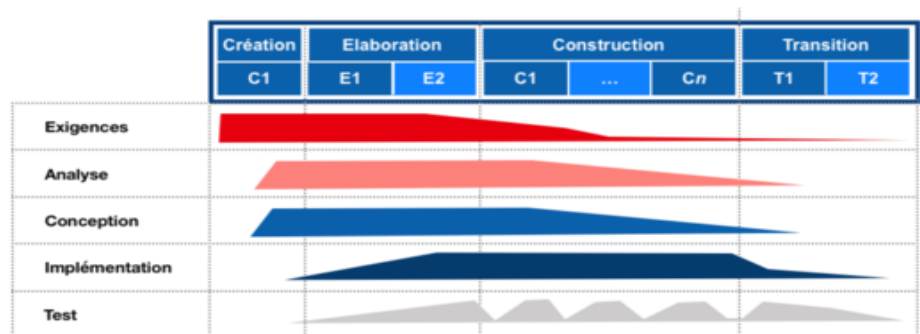


Figure 2 : processus unifié (cycle de vie)

2.6 Présentation du Processus 2TUP

Le 2TUP est un modèle de développement de logiciels qui a été conçu pour améliorer la qualité du processus unifié (UP) original.

Le 2TUP propose un cycle de développement en Y, qui dissocie les aspects techniques des aspects fonctionnels. Il commence par une étude préliminaire qui consiste essentiellement à identifier les acteurs qui vont interagir avec le système à construire, les messages qu'échangent les acteurs et le système, à produire le cahier des charges et à modéliser le contexte.

Le processus s'articule ensuite autour de trois phases essentielles : une phase reliée à la branche fonctionnelle, une phase reliée à la branche technique et une phase de réalisation.

La branche fonctionnelle capitalise la connaissance du métier de l'entreprise. Cette branche capture des besoins fonctionnels, ce qui produit un modèle focalisé sur le métier des utilisateurs finals.

La branche technique capitalise un savoir-faire technique et/ou des contraintes techniques, Les techniques développées pour le système le sont indépendamment des fonctions à réaliser.

La phase de réalisation consiste à réunir les deux branches en vue de mener une conception applicative et assurer la livraison d'une solution adaptée aux besoins fonctionnels et techniques.

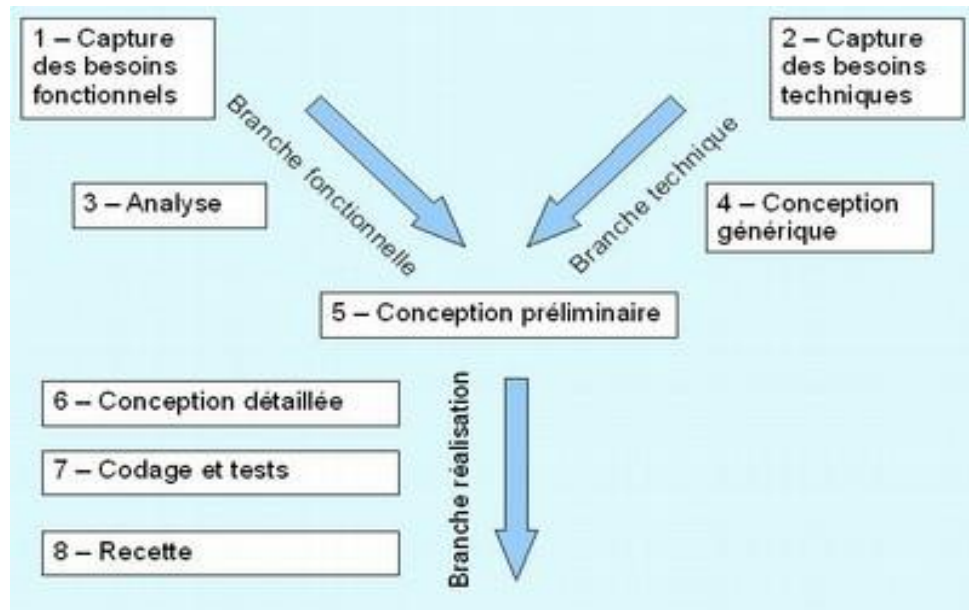


Figure 3 : processus 2TUP

2.7 2TUP et UML

2.7.1 Description d'UML

Le processus 2TUP se base sur l'utilisation continue d'UML tout au long du cycle de développement. Les divers diagrammes d'UML, en raison de leur simplicité et de leur clarté, offrent une modélisation adéquate du système à chaque phase du processus.

UML se définit comme étant un langage de modélisation graphique et textuel destiné à comprendre et décrire des besoins, spécifier, concevoir des solutions et communiquer des points de vue.

UML unifie à la fois les notations et les concepts orientés objet. Il ne s'agit pas d'une simple notation, mais les concepts transmis par un diagramme ont une sémantique précise et sont porteurs de sens au même titre que les mots d'un langage, c'est pour cela qu'UML est présenté parfois comme une méthode alors qu'il ne l'est absolument pas.

UML unifie également les notations nécessaires aux différentes activités d'un processus de développement et offre, par ce biais, le moyen d'établir le suivi des décisions prises, depuis la définition des besoins jusqu'au codage (Roques, 2006).

2.7.2 Catalogue des diagrammes d'UML

La section ci-dessous est consacrée à la présentation sommaire des différents diagrammes UML qui vont être utilisés tout au long de notre projet.

Le diagramme des cas d'utilisation représente la structure des fonctionnalités nécessaires aux utilisateurs du système. Il est normalement utilisé lors des étapes de capture des besoins fonctionnels et techniques.

Le diagramme d'activités représente les règles d'enchaînement des activités et actions dans le système. Il peut être assimilé comme un algorithme mais schématisé.

Le diagramme de package présent depuis UML 2.0, ce diagramme modélise des catégories cohérentes entre elles, pour un souci de partage des rôles. Correspond à l'étape de modélisation des différents scénarios d'un cas d'utilisation.

Le diagramme de classes est l'un des diagrammes les plus importants dans un développement orienté objet. Sur la branche fonctionnelle, ce diagramme est prévu pour modéliser la structure des entités manipulées par les utilisateurs. En conception, le diagramme de classes représente la structure d'un code orienté objet.

Le diagramme de séquence représente les échanges de messages entre objets, dans le cadre d'un fonctionnement particulier du système.

Le diagramme d'états représente le cycle de vie d'un objet. Il spécifie les états possibles d'une classe et leur enchaînement. Ce diagramme est utilisé lors des étapes d'analyse et de conception.

Chapitre 3 : Etude de l'existant

L'étude de l'existant est une phase importante du processus de développement. Elle consiste à examiner les systèmes existants et les solutions similaires pour mieux comprendre les attentes des utilisateurs, les exigences du marché et les meilleures pratiques. Cette étude permet de déterminer ce qui fonctionne bien et ce qui peut être amélioré dans les solutions existants, ce qui peut aider à orienter la conception et le développement de notre solution future.

3.1 Constatations

Dans cette partie, nous allons voir d'une part les modalités de fonctionnement actuel des activités de prestations de services des ouvriers du BTP et d'autre part recueillir les besoins des employeurs demandeurs de ces prestations de service.

Au Congo, les ouvriers ont souvent des marchés dans la plupart des cas de bouche à oreille ou par recommandation de certains employeurs. Certes ces pratiques sont relativement efficaces, mais elles limitent le champ d'application de leur travail.

Pour les ouvriers, la reconnaissance du travail accompli est souvent restreinte à la zone où les prestations de services sont fournies, et la quête de nouvelles opportunités sur le marché n'est pas aisément accessible.

Du côté des employeurs, les sélections d'ouvriers sont fréquemment influencées par les recommandations plutôt que par les compétences avérées, et il n'existe pas de moyen de publier des annonces de demande de service d'emploi d'ouvrier.

Étant donné ce déséquilibre entre l'offre et la demande sur le marché local du BTB, certains problèmes se posent, tels que des travailleurs qui ne sont pas en mesure de cibler un large éventail de marchés, ainsi que des employeurs qui se limitent à leur environnement proche pour trouver des ouvriers. De plus, d'autres limitations comprennent :

- L'absence de garantie concernant le profil des ouvriers.
- L'absence d'un portfolio pour les ouvriers.
- Le manque de possibilité pour les employeurs de soumettre des appels d'offres pour des chantiers ou des projets de rénovation.

3.2 Solutions existantes

Il existe de nombreuses solutions plateforme de mise en relation entre les ouvriers et les employeurs dans le monde. Parmi lesquels figurent : Mon-ouvrier, Habitatpresto

- **Mon-ouvrier**

✓ **Historiques :**

Existant depuis 25 ans, mon-ouvrier.com est une plateforme exerçant dans le domaine du bâtiment et de la rénovation. Leur but est d'accompagner et de réaliser des travaux d'intérieurs et d'extérieurs [4].

✓ Fonctionnalités :

✓ Demande de devis

Le processus de demande de devis se fait comme suit : l'utilisateur doit d'abord saisir les informations suivantes : le nom, l'email, le téléphone, l'adresse, ville, statut, service, description du projet

Figure 4 : demande de devis mon-ouvrier

✓ Services proposés

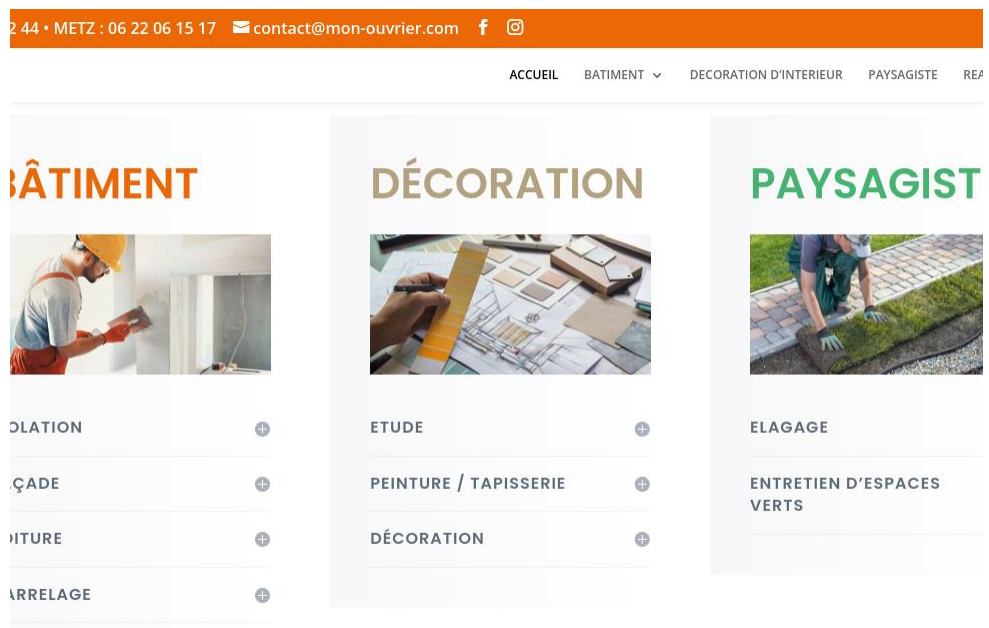


Figure 5 : services proposés mon-ouvrier

- **Habitatpresto**

- ✓ **Historique :**

Habitatpresto est une société fondée en 2006 par la visionnaire Emeline Mann, persuadée qu'en simplifiant la recherche de professionnels pour réaliser ses travaux était indispensable pour mener sereinement ses projets. Le service a très rapidement attiré de nombreux employeurs et professionnels, voyant une réelle opportunité de collaborer efficacement.

Nous avons parcouru un long chemin depuis ! La société Habitatpresto a été rachetée en 2013 par MyBestPro, et notre équipe s'est fortement agrandie : nous sommes aujourd'hui 50 employés, fiers de participer au bien-être de votre foyer ! [5]

- ✓ **Fonctionnalité :**

- ✓ Faire une demande de devis

Le principe ici est de trouver un professionnel de confiance pour les travaux c'est pourquoi il faut choisir parmi la liste des travaux proposés qui sont : travaux les plus populaires, aménagement intérieur, architecture – expertise, assainissement – terrassement, chauffage – chaudière, chauffage et énergie renouvelable, construction – agrandissement, électricité – courant faible, étanchéité – humidité – ventilation, fenêtre – volet, isolation – plâtre, jardin – clôture et portail, maçonnerie – démolition.



Figure 6 : demande de devis Habitatpresto

- ✓ Création de compte

Permet d'accéder à un espace personnel pour gérer les professionnels qui vous ont contactés, consulter les profils correspondant à vos besoins, trouver

des inspirations liées à vos projets. Pour cela il suffit d'inscrire votre email et votre mot de passe.

Figure 7 : création d'un compte

- ✓ Compte professionnel pour les ouvriers
Ce compte permet aux professionnels du bâtiment de trouver de nouveaux chantiers en toute simplicité, il suffit de choisir son profil et sa ville pour trouver une annonce.

Figure 8 : création d'un compte professionnel Habitatpresto

Tableau comparatif

Le tableau suivant représente une comparaison entre les différentes solutions existante :

Solutions	Zone géographique d'utilisation	Avantages	Inconvénients
Mon-ouvrier	France	▪ Permet de contacter l'ouvrier ▪ Voir les réalisations effectuées ▪ Permet d'établir un devis ▪ Proposition des espèces dans le domaine du bâtiment	▪ Pas de moyens de connaître le profil de l'ouvrier ▪ Impossibilité de voir les réalisations de l'ouvrier
Habitatpresto	France	▪ Permet d'établir un devis ▪ Créer un projet de rénovations ▪ Voir les profils des ouvriers ▪ Voir les témoignages des employeurs ▪ Possibilité de trouver un chantier pour les ouvriers ▪ Propositions des conseils dans le BTP	▪ Pas moyens de connaître le profil de l'ouvrier ▪ Impossibilité de voir les réalisations de l'ouvrier

Tableau 2 : Tableau comparatif

3.3 Proposition de solution

Les solutions envisageables peuvent répondre aux besoins des employeurs et ouvriers sont les suivants :

- S'inspirer des solutions existantes ;
- création d'un annuaire de domaine BTP qui permettra de référencer tous les ouvriers ;
- mettre en place une application mobile qui permettra de mettre en relation les employeurs et les ouvriers.

3.4 Choix de la solution

Aux vues des inconvénients précités, nous proposons de mettre en place une application mobile en ajoutant les fonctionnalités qui sont les suivants :

- faciliter la recherche des profils des ouvriers,
- permettre aux ouvriers de trouver rapidement des offres,
- améliorer le suivi des offres,
- permettre aux ouvriers de faire connaître leurs réalisations,
- permettre l'évaluation des ouvriers à la fin de chaque contrat,
- permettre au particulier de signaler un ouvrier,
- permettre au gestionnaire d'assurer la création et le blocage temporaire ou définitif des comptes ouvriers.

Chapitre 4 : Analyse

4.1 Analyse des besoins

L'analyse des besoins est un processus crucial dans le développement de logiciels de qualité. Elle consiste à comprendre les exigences et les attentes des utilisateurs pour déterminer les fonctionnalités nécessaires de la solution future.

- **Besoins fonctionnels**

Les besoins fonctionnels représentent les fonctionnalités du système. Notre système fournira aux utilisateurs les fonctionnalités suivantes : l'authentification, la gestion du référentiel des emplois BTP, l'annuaire des ouvriers, la gestion des qualifications des ouvriers, la gestion des offres et réservations, la gestion des discussions, et la gestion des comptes utilisateurs.

La gestion du référentiel des emplois du BTP vise à intégrer dans la solution la définition et la classification des différents emplois et métiers qui existent dans le domaine BTP au Congo. Ce référentiel est destiné enregistrer la diversité des emplois présents dans le secteur, en fournissant des descriptions détaillées des compétences, des tâches et des responsabilités associées à chaque catégorie d'ouvrier.

L'annuaire des ouvriers permettra de référencer l'identité des ouvriers qui s'inscriront sur la plateforme en vue de répondre aux offres d'emploi des employeurs. A cet annuaire sera associée la gestion des qualifications des ouvriers basée sur le référentiel des emplois du BTP.

La gestion des qualifications des ouvriers implique le suivi et la maintenance des informations relatives aux compétences, aux certifications, aux formations et aux expériences professionnelles acquises par les ouvriers référencés dans la plateforme.

La gestion des offres et des travaux permettra de prendre en compte les offres de travaux émis par les employeurs.

La gestion des discussions sera une messagerie d'échange d'information entre acteurs de la plateforme.

- **Besoins techniques**

Il s'agit des besoins qui caractérisent le système. Ce sont des besoins non fonctionnels, besoins en matière de performance, de type de matériel, de confidentialité, de conception des interfaces homme-machines.

Dans cette optique, le système à concevoir devra présenter les attributs suivants : convivialité, traitement rapide, facilité d'utilisation et sécurité.

4.2 Spécification fonctionnelle

4.2.1 Identification des acteurs

Un acteur joue un rôle important dans le système. Il peut être une personne, un matériel ou un dispositif externe. Une même personne physique peut se comporter en autant d'acteurs différents que le nombre de rôles qu'elle joue vis-à-vis du système.

Les acteurs de notre système sont :

- L'ouvrier est un utilisateur qui s'inscrira lui-même. Il lui sera exposé comme fonctionnalité principale la recherche des offres.
- L'employeur est un utilisateur qui s'inscrira lui-même. Il lui sera exposé comme fonctionnalité principale la recherche des ouvriers du BTP.
- Le gestionnaire qui est un utilisateur qui sera créé par l'administrateur du système, il gèrera les employeurs et les ouvriers et recevra les demandes de création de compte et les plaintes des employeurs.
- L'administrateur est le super utilisateur du système. Il dispose de tous les droits d'accès de gestion des profils des utilisateurs et des utilisateurs sur le système. Mais il n'aura aucun accès sur les fonctionnalités métier de confrontation de l'offre et la demande de réalisation des travaux.

4.2.2 Diagramme de contexte statique

Le diagramme de contexte statique modélise la vue statique d'un système, c'est-à-dire la vue du système à un moment donné sans prendre en compte les évolutions temporelles.



Figure 9 : diagramme de contexte statique

4.2.3 Identification des cas d'utilisation

Un cas d'utilisation (use case) modélise un service rendu par le système. Il représente un ensemble de séquences d'actions réalisées par le système et produisant un résultat observable intéressant pour un acteur particulier.

On ce qui concerne ce projet nous avons retenu les cas d'utilisation suivants :

- S'authentifier
- Ajouter un domaine
- Rechercher un domaine
- Supprimer un domaine

- Modifier un domaine
- Ajouter travaux
- Rechercher travaux
- Supprimer travaux
- Modifier travaux
- Ajouter un métier
- Rechercher un métier
- Modifier un métier
- Supprimer un métier
- Publier une offre
- Supprimer une offre
- Annuler une offre
- Modifier une offre
- Réserver un ouvrier
- Rédiger une appréciation
- Annuler une réservation
- Commenter une offre
- Rechercher un commentaire
- Rechercher une réservation
- Rechercher une appréciation
- Rechercher une offre
- Ajouter une expérience
- Ajouter une formation
- Ajouter une qualification
- Ajouter une compétence
- Modifier une expérience
- Modifier une formation
- Modifier une qualification
- Modifier une compétence
- Supprimer une expérience
- Supprimer une formation
- Supprimer qualification
- Supprimer une compétence
- Rechercher une expérience
- Rechercher une formation
- Rechercher une qualification
- Rechercher une compétence

- Créer un compte admin
- Créer un compte ouvrier
- Créer un compte employeur
- Créer un compte gestionnaire
- Valider un compte
- Archiver un compte
- Désactiver un compte
- Supprimer un compte
- Activer un compte
- Bloquer temporairement un compte
- Bloquer définitivement un compte
- Rechercher un compte
- Modifier un compte
- Signaler un compte

4.2.4 Classification par package

Un package (ou paquetage en français) en UML sert à grouper des éléments en un ensemble cohérent. Il peut contenir :

- des éléments d'un modèle,
- des diagrammes qui représentent les éléments du modèle, d'autres packages.

Ainsi nous avons 5 packages :

- ✓ Référentiel du domaine du BTP
- ✓ Gestion des offres et réservations
- ✓ Gestion des qualifications des ouvriers
- ✓ Gestion des discussions
- ✓ Gestions des comptes utilisateurs

Nom du Package	Cas d'utilisation	Acteurs
Référentiel du domaine du BTP	▪ Ajouter un domaine ▪ Rechercher un domaine ▪ Supprimer un domaine ▪ Modifier un domaine ▪ Ajouter travaux ▪ Rechercher travaux ▪ Supprimer travaux ▪ Modifier travaux ▪ Ajouter un métier ▪ Rechercher un métier ▪ Modifier un métier ▪ Supprimer un métier ▪ S'authentifier	▪ Le gestionnaire
Gestion des offres et réservations	▪ Publier une offre ▪ Supprimer une offre ▪ Annuler une offre ▪ Modifier une offre ▪ Réserver un ouvrier ▪ Rédiger une appréciation	▪ Employeur ▪ Ouvrier

	▪ Annuler une réservation ▪ Commenter une offre ▪ Rechercher un commentaire ▪ Rechercher une réservation ▪ Rechercher une appréciation ▪ Rechercher une offre ▪ S'authentifier	
Gestions des qualifications des ouvriers	▪ Ajouter une expérience ▪ Ajouter une formation ▪ Ajouter une qualification ▪ Ajouter une compétence ▪ Modifier une expérience ▪ Modifier une formation ▪ Modifier une qualification ▪ Modifier une compétence ▪ Supprimer une expérience ▪ Supprimer une formation ▪ Supprimer qualification ▪ Supprimer une compétence ▪ Rechercher une expérience ▪ Rechercher une formation ▪ Rechercher une qualification ▪ Rechercher une compétence	▪ Ouvrier
Gestion des discussions	▪ Supprimer un message ▪ Archiver un message ▪ Envoyer un message ▪ Envoyer photo ▪ Envoyer vidéo ▪ Envoyer un fichier ▪ S'authentifier	▪ Employeur ▪ Ouvrier
Gestion des comptes utilisateurs	▪ Créer un compte admin ▪ Créer un compte ouvrier ▪ Créer un compte employeur ▪ Créer un compte gestionnaire ▪ Valider un compte ▪ Archiver un compte ▪ Désactiver un compte ▪ Supprimer un compte ▪ Activer un compte ▪ Bloquer temporairement un compte ▪ Bloquer définitivement un compte ▪ Rechercher un compte ▪ Modifier un compte ▪ Signaler un compte ▪ S'authentifier	▪ Administrateur ▪ Ouvrier ▪ Employeur

Tableau 3 : Classification par package

4.2.5 Relation entre cas d'utilisation (Use case)

En langage UML, une relation est une connexion entre des éléments de modèle. Une relation UML est un type d'élément de modèle qui ajoute une sémantique à un modèle en définissant la structure et le comportement entre les éléments de modèle.

Pour garantir que le diagramme de cas d'utilisation soit immédiatement compréhensible pour tout un chacun, on utilise des briques de base standardisées pour la représentation. Il y a en premier lieu trois éléments principaux :

- **Acteur** : les acteurs, que ce soit des personnes ou des systèmes, sont représentés par des bonhommes en fil de fer.
- **Système** : le système auquel se rapporte le cas d'utilisation est représenté par un rectangle.
- **Use Case** : le cas d'utilisation lui-même est représenté par une ellipse, dans laquelle il y a généralement une courte phrase décrivant le processus.

Les relations entre ces éléments sont représentées par des lignes de connexion, appelées associations. Ces relations comprennent la relation d'association, de généralisation, d'inclusion et d'extension.

En UML, la relation d'association est un type de lien entre des objets ou classes d'objets qui représente une connexion logique entre eux. Les relations d'association décrivent comment les objets travaillent ensemble et comment ils se transmettent des données et des informations entre eux.

La relation de généralisation, également appelée héritage ou généricité, représente une relation de parenté entre des classes ou des objets. Elle est utilisée pour modéliser l'idée d'une classe générale qui est dérivée en une ou plusieurs classes spécifiques, appelées classes filles.

La relation d'inclusion est une relation dans laquelle un cas d'utilisation (le cas d'utilisation de base) inclut les fonctionnalités d'un autre cas d'utilisation (le cas d'utilisation inclus). La relation d'inclusion prend en charge la réutilisation des fonctionnalités dans un modèle de cas d'utilisation.

La relation d'extension est utilisée pour spécifier qu'un cas d'utilisation (l'extension) étend le comportement d'un autre cas d'utilisation (la base). Ce type de relation révèle des détails sur un système ou une application qui est généralement masqué dans un cas d'utilisation.

4.2.6 Classification des cas d'utilisation (priorités, risque et itération)

Dans le cadre d'un développement itératif et incrémental, il est très utile de recourir au découpage en cas d'utilisation pour définir les itérations. A cet effet, il convient en premier lieu d'identifier les cas d'utilisation les plus critiques en matière de gestion des risques. Ces cas d'utilisation doivent être traités en priorité pour lever au plus tôt les risques majeurs.

Cas d'utilisation	Risque	Priorité	Itération
S'authentifier	Haut	Haut	1
Ajouter un domaine	Moyen	Moyenne	2
Rechercher un domaine	Faible	Basse	3
Supprimer un domaine	Faible	Basse	3
Modifier un domaine	Moyen	Moyenne	2
Ajouter travaux	Moyen	Moyenne	2
Rechercher travaux	Faible	Basse	3
Supprimer travaux	Faible	Basse	3
Modifier travaux	Moyen	Moyenne	2
Ajouter un métier	Moyen	Moyenne	2
Rechercher un métier	Faible	Basse	3
Modifier un métier	Moyen	Moyenne	2
Supprimer un métier	Faible	Basse	3
Publier une offre	Faible	Basse	5
Supprimer une offre	Faible	Basse	6
Annuler une offre	Faible	Basse	6
Modifier une offre	Faible	Basse	6
Réserver un ouvrier	Faible	Basse	5
Rédiger une appréciation	Faible	Basse	7
Annuler une réservation	Faible	Basse	7
Commenter une offre	Faible	Basse	8
Rechercher un commentaire	Faible	Basse	8
Rechercher une réservation	Faible	Basse	8
Rechercher une appréciation	Faible	Basse	8
Rechercher une offre	Faible	Basse	4
Ajouter une expérience	Faible	Basse	4
Ajouter une formation	Faible	Basse	4
Ajouter une qualification	Faible	Basse	4
Ajouter une compétence	Faible	Basse	4
Modifier une expérience	Faible	Basse	4
Modifier une formation	Faible	Basse	4
Modifier une qualification	Faible	Basse	4
Modifier une compétence	Faible	Basse	4
Supprimer une expérience	Faible	Basse	4
Supprimer une formation	Faible	Basse	4
Supprimer qualification	Faible	Basse	4
Supprimer une compétence	Faible	Basse	4
Rechercher une expérience	Faible	Basse	4
Rechercher une formation	Faible	Basse	4
Rechercher une qualification	Faible	Basse	4
Rechercher une compétence	Faible	Basse	4
Supprimer un message	Faible	Basse	4
Archiver un message	Faible	Basse	5
Envoyer un message	Faible	Basse	5
Envoyer photo	Faible	Basse	5
Envoyer vidéo	Faible	Faible	5
Envoyer un fichier	Faible	Basse	5

Cas d'utilisation	Risque	Priorité	Itération
Créer un compte admin	Moyen	Moyenne	2
Créer un compte ouvrier	Moyen	Moyenne	2
Créer un compte employeur	Moyen	Moyenne	2
Créer un compte gestionnaire	Moyen	Moyenne	2
Valider un compte	Moyen	Moyenne	2
Archiver un compte	Faible	Basse	3
Désactiver un compte	Faible	Basse	3
Supprimer un compte	Faible	Basse	3
Activer un compte	Faible	Basse	3
Bloquer temporairement un compte	Faible	Basse	3
Bloquer définitivement un compte	Faible	Basse	3
Rechercher un compte	Faible	Basse	3
Modifier un compte	Faible	Basse	3
Signaler un compte	Faible	Basse	3

Tableau 4 : classification des cas d'utilisation (risque, priorité, itération)

4.3 Spécifications détaillées

- Description textuelle des cas d'utilisation

La description textuelle des cas d'utilisations décrit le scénario dans lequel un utilisateur interagit avec le système pour accomplir une tâche spécifique.

Cas d'utilisation "Ajouter un domaine BTP"

SOMMAIRE D'IDENTIFICATION		
Titre	Ajouter un domaine BTP	
Objectif	Permettre au gestionnaire de gérer les domaines disponibles pour les travaux.	
Déclencheur	Activation de la commande « Ajouter un domaine BTP »	
Dépendance	Cas d'utilisation inclus	S'authentifie, Rechercher
	Cas d'utilisation optionnel	
Mode	Interactif	
Acteur(s)	Acteur principal	Gestionnaire
	Acteur secondaire	
Fréquence d'utilisation	A la demande	
DESCRIPTION DES ENCHAINEMENTS		
Pré condition(s)		Post condition(s)
Le gestionnaire doit être connecté à l'application et avoir les autorisations nécessaires pour gérer les domaines.		Le nouveau domaine est ajouté à la liste des domaines disponibles dans la base la donnée
SCENARIO NOMINAL		
Etapes	Activités	Scenario alternatif

1	Le gestionnaire active la commande « ajouter un domaine BTP »	
2	Le gestionnaire accède la section “Ajouter un domaine” dans l’application	
3	Le gestionnaire saisit les informations nécessaires pour décrire le nouveau domaine	SA3
4	Le gestionnaire soumet la demande d’ajout du nouveau domaine	
5	Le système vérifie les informations et, si elles sont valides	SA5.1, SA5.2, SA5.3, SA5.4
6	Le système enregistre le nouveau domaine dans la liste des domaines disponible	
7	Le gestionnaire reçoit une notification indiquant que le nouveau domaine a été ajouté avec succès	
SCENARIO ALTERNATIF		
SA3- Le système affiche un message d’erreur et le cas d’utilisation reprend à l’étape 3 du scénario nominal afin que le gestionnaire corrige les erreurs de saisie SA5.1- Après recherche, si le domaine existe déjà dans la base de données, le système affiche un message d’erreur indiquant que le domaine existe déjà SA5.2 –le gestionnaire peut choisir de modifier les détails du domaine ou d’annuler l’ajout du domaine SA5.3- Si le gestionnaire décide de modifier les détails du domaine, le système met à jour les informations dans la base de données SA5.4- Si le gestionnaire décide d’annuler l’ajout du domaine, le système retourne à l’écran précédent		
SCENARIO D’EXCEPTION		
E1- Problème de connexion à la base de données. e1- le système ne peut pas se connecter à la base de données pour vérifier ou ajouter les informations, un message d’erreur sera affiché. E2- Autorisation insuffisante e1- Si le gestionnaire n’a pas les autorisations nécessaires pour ajouter un nouveau domaine, le système affichera un message d’erreur indiquant qu’il n’a pas les autorisations nécessaires pour ajouter un nouveau domaine.		
REMARQUES PARTICULIERES SUR LES FLUX D’INFORMATIONS		
▪ Données en entrée : le nom du domaine à ajouter ▪ Données en sortie : confirmation de l’ajout du domaine.		

Figure 10 : Ajouter un domaine

Diagramme de cas d’utilisation “Créer un compte ”

SOMMAIRE D’IDENTIFICATION		
Titre	Créer compte	
Objectif	Permettre à l’utilisateur de créer un compte dans le système	
Déclencheur	Activation de la commande « créer compte ouvrier »	
Dépendance	Cas d’utilisation inclus	S’authentifier, Recherche
	Cas d’utilisation optionnel	
Mode	Interactif	
Acteur(s)	Acteur principal	Ouvrier

	Acteur secondaire	L'administrateur
Fréquence d'utilisation	A la demande	
DESCRIPTION DES ENCHAINEMENTS		
Pré condition(s)		Post condition(s)
L'application est lancée et l'ouvrier doit s'authentifier.		Le compte ouvrier est créé avec succès et doit être validé par l'administrateur.
SCENARIO NOMINAL		
Etapas	Activités	Scenario alternatif
1	L'ouvrier active la commande « Créer compte ouvrier »	
2	Le système affiche un formulaire de création de compte en fournissant ses informations personnelles	
3	L'ouvrier saisi les informations nécessaires	SA3
	L'administrateur vérifie le compte ouvrier et le valide ou le rejette	
4	Le système enregistre le compte dans la base de données	SA4.1 , SA2
5	Le système enregistre les informations du compte ouvrier et génère un message de confirmation	
SCENARIO ALTERNATIF		
SA3- Si l'un des champs de saisie contient des informations invalides le système affiche un message d'erreur et demande à l'utilisateur de corriger les champs invalides avant de pouvoir continuer, le scénario reprend à l'étape 3.		
SA4.1- Après recherche, si compte de l'utilisateur existe déjà dans la base de données, le système affiche un message d'erreur indiquant que le compte existe déjà.		
SA4.2 –l'utilisateur peut choisir de modifier les détails du compte ou d'annuler la création de compte		
SCENARIO D'EXCEPTION		
E1- Si le système rencontre une erreur technique lors de la création de compte, un message d'erreur est affiché à l'utilisateur pour l'informer de l'erreur.		
REMARQUES PARTICULIERES SUR LES FLUX D'INFORMATIONS		
- Données en entrée : les informations personnelles de l'ouvrier. - Données en sortie : le message de confirmation de l'inscription		

Figure 11 : Créer un compte

4.4 Diagramme de séquence

Les diagrammes de séquences sont la représentation graphique des interactions entre les acteurs et le système.

Il permet de représenter des échanges entre différents objets et acteurs du système en fonction du temps.

La dimension verticale du diagramme représente le temps, permettant de visualiser l'enchaînement des actions dans le temps et de spécifier la naissance et la destruction de l'objet. Les périodes d'activités des objets sont symbolisées par des rectangles et ces objets dialoguent à l'aide de messages.

Diagramme de séquence “ Ajouter un domaine ”

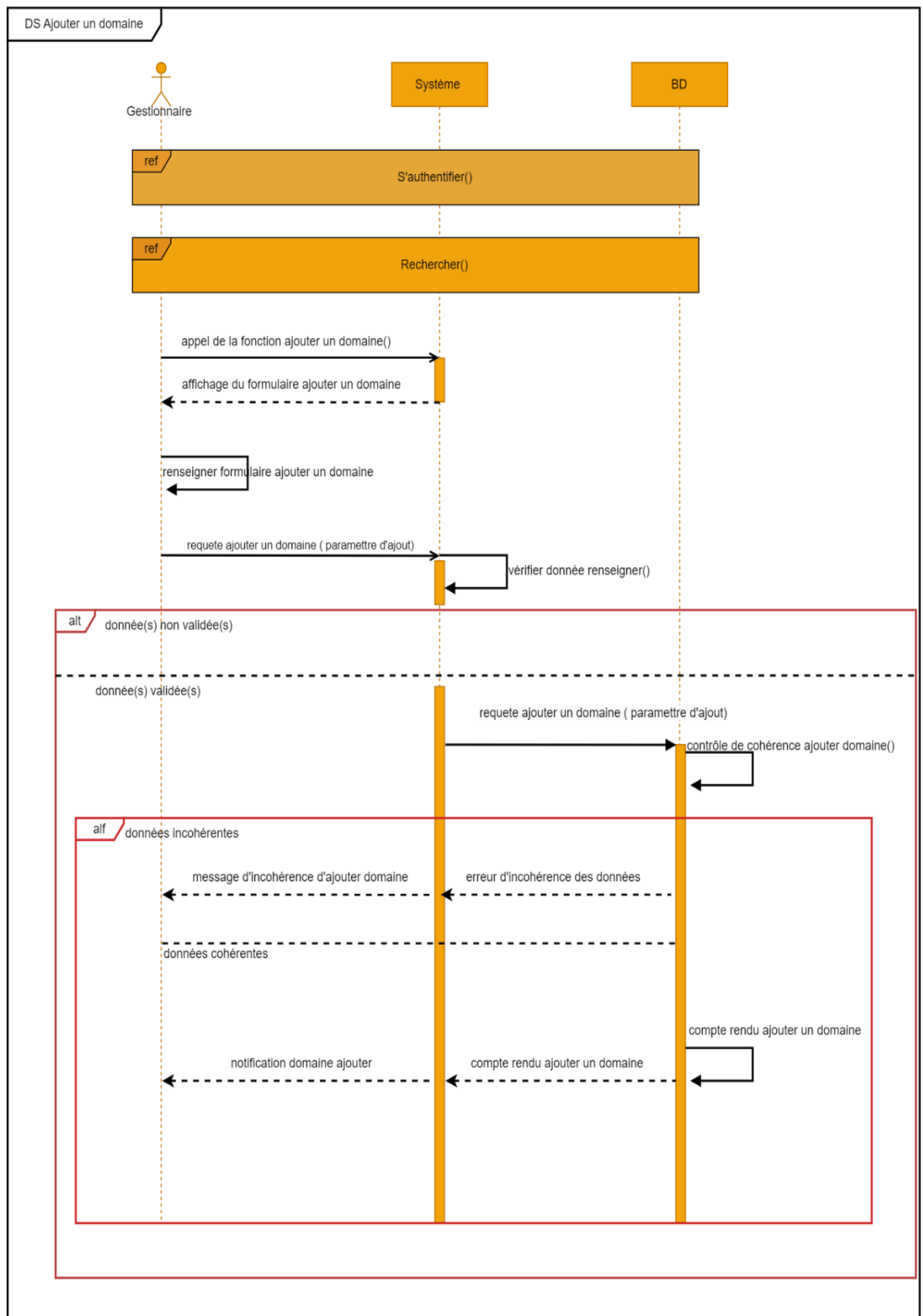


Figure 12 : Diagramme de séquence (Ajouter un domaine)

Diagramme de séquence “ Créer un compte”

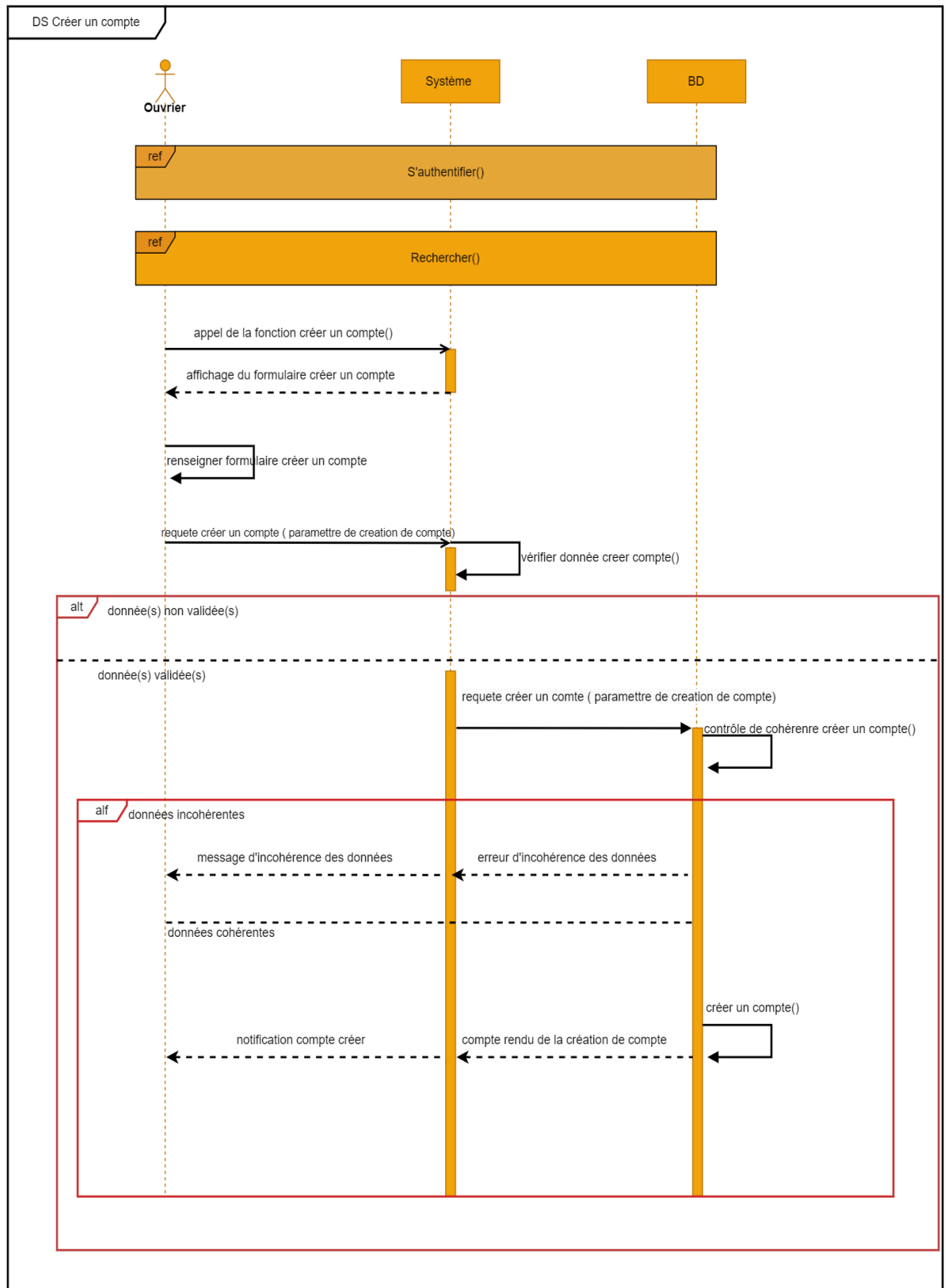


Figure 13 : Diagramme de séquence (Créer un compte)

4.5 Diagramme d'activité

Le diagramme d'activité est un outil pour illustrer les séquences d'actions, les flux de travail et les processus décisionnels qui caractérisent notre système. Il offre une vue d'ensemble structurée des différentes étapes et interactions, ce qui facilite la compréhension pour les parties prenantes externes.

Diagramme d'activité "Ajouter un domaine"

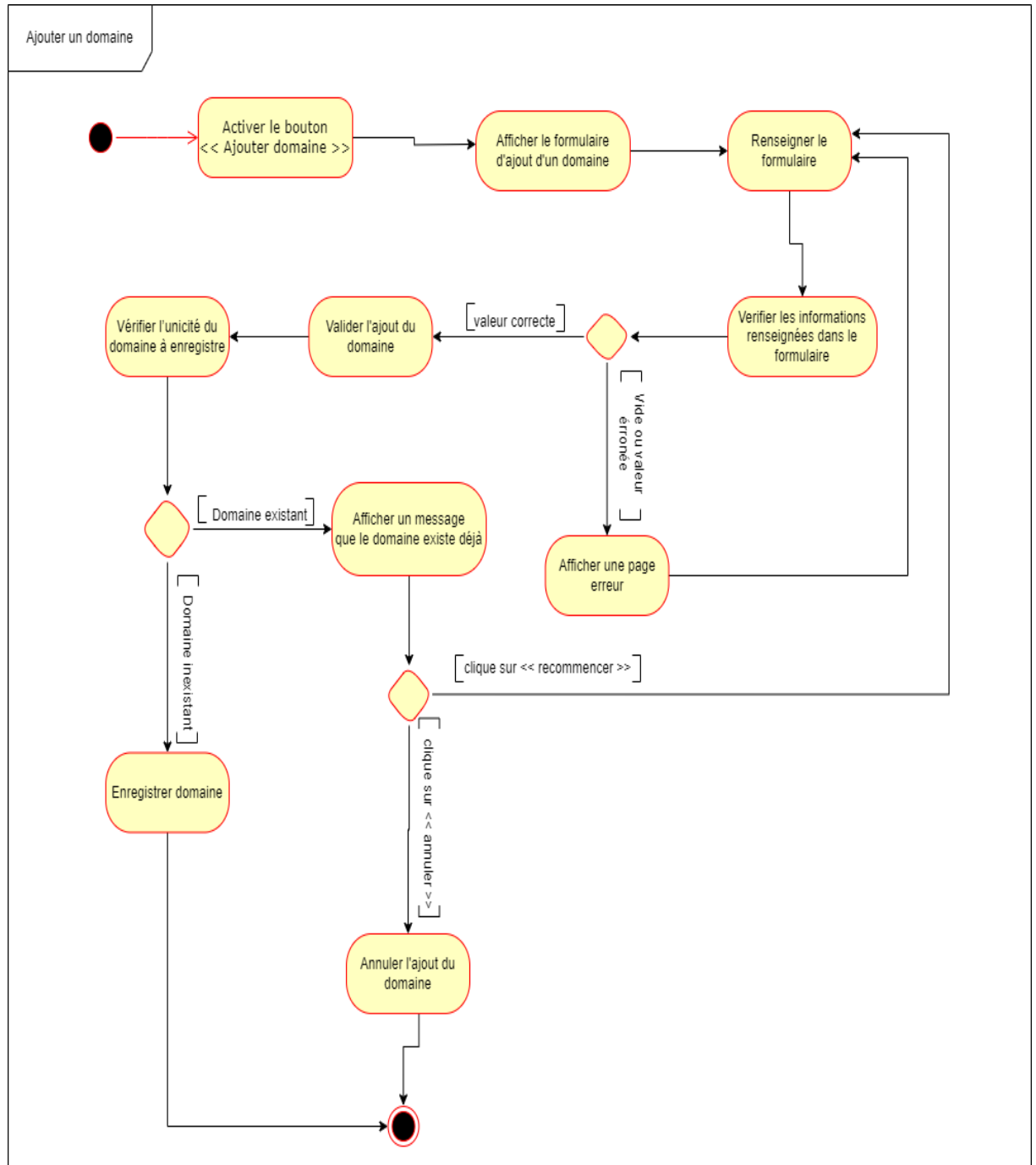


Figure 14 : Diagramme d'activité ajouter un domaine

■ Diagramme d'activité << **Créer un compte** >>

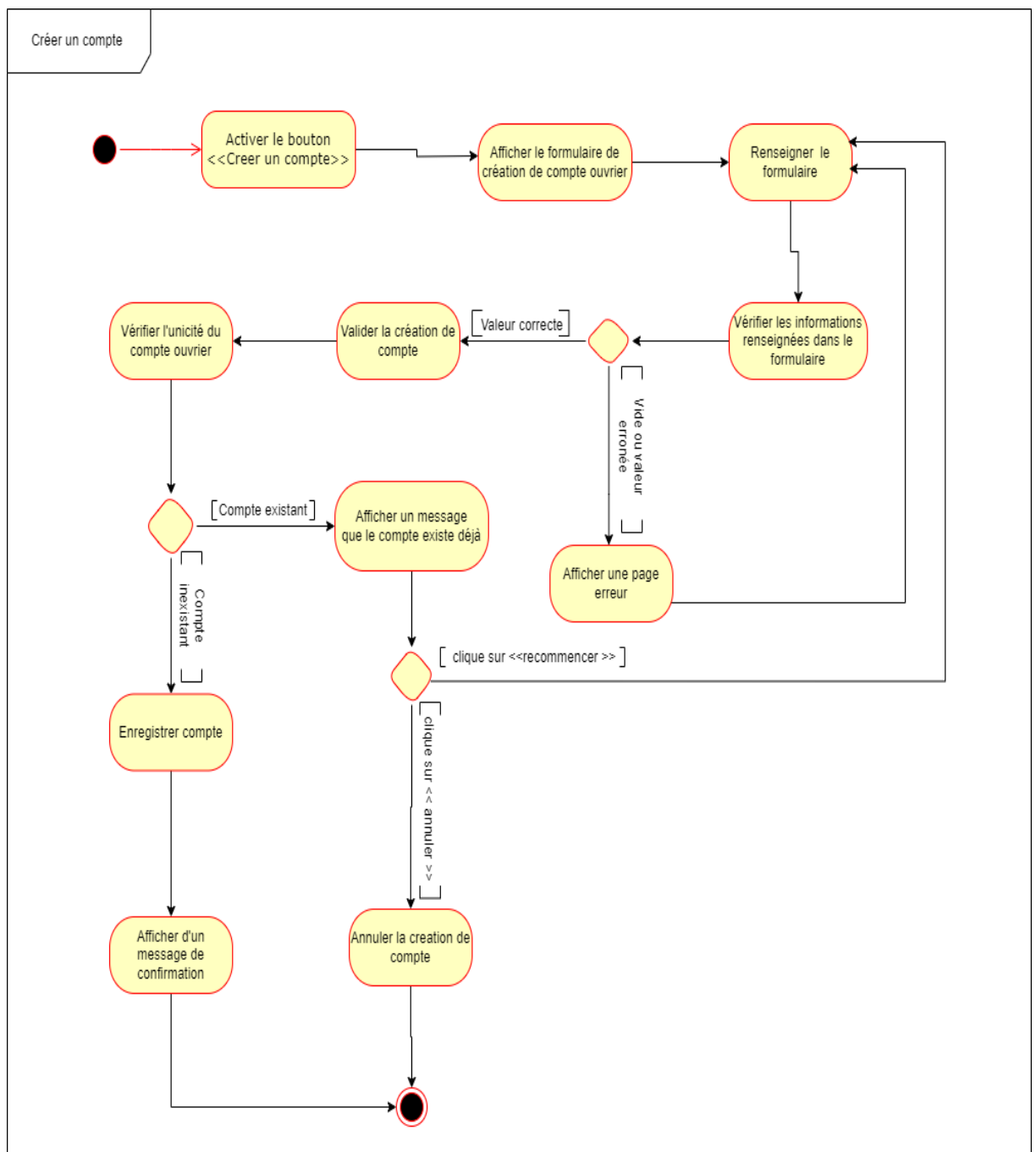


Figure 15 : Diagramme d'activité créer un compte

Chapitre 5 : Conception

La conception consiste à élaborer une solution technique pour répondre aux besoins fonctionnels et techniques des utilisateurs identifiés lors l'analyse.

5.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation est un diagramme UML (Unified Modeling Language) qui permet de représenter graphiquement les interactions entre les utilisateurs du système.

5.1.1 Diagramme de cas d'utilisation de gestion du Référentiel du domaine du BTP

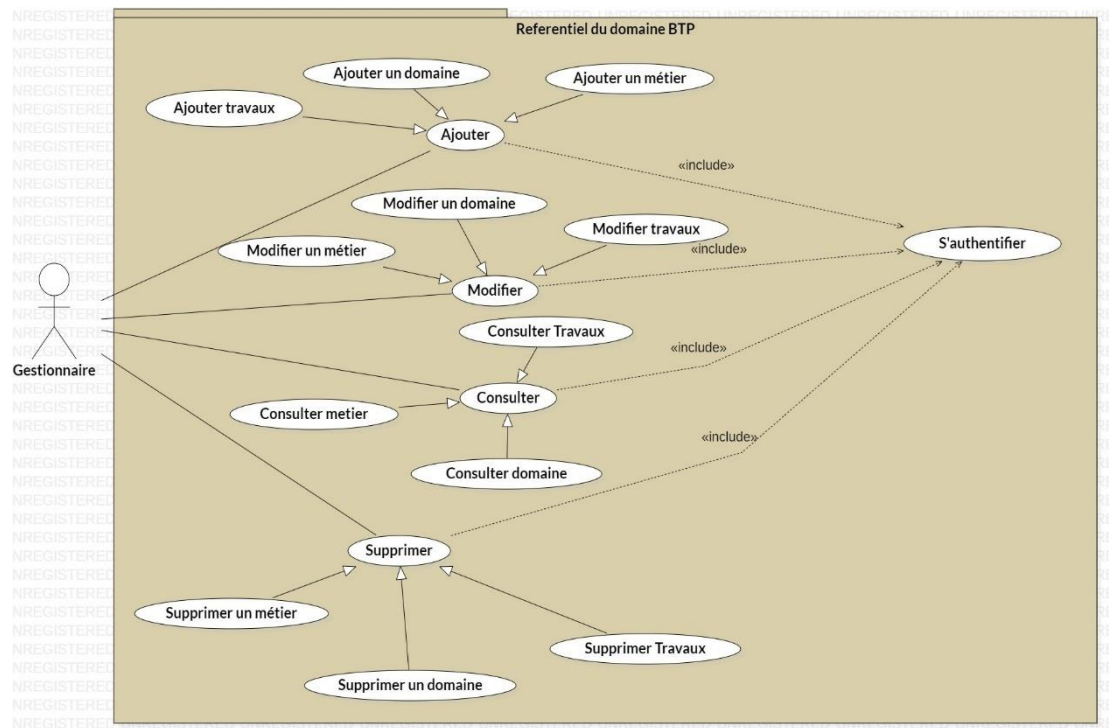


Figure 16 : diagramme de cas d'utilisation (référentiel des domaines BTP)

5.1.2 Diagramme de cas d'utilisation de gestion des offres et réservations

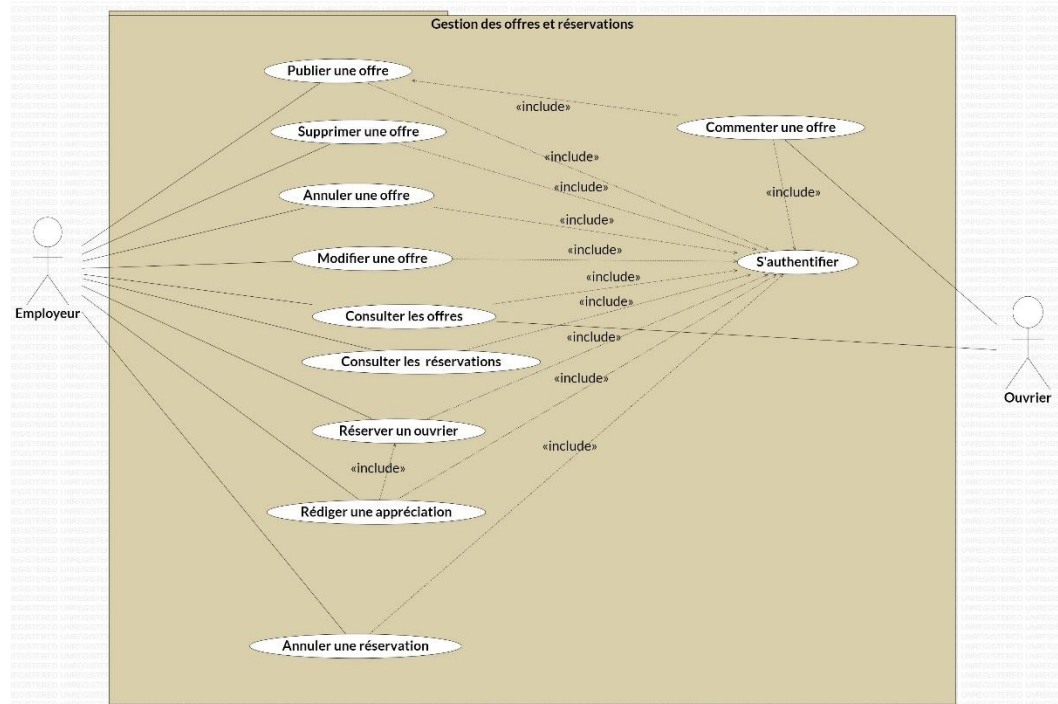


Figure 17 : diagramme de cas d'utilisation (gestion des offres et réservation)

5.1.3 Diagramme de cas d'utilisation de gestion des qualifications

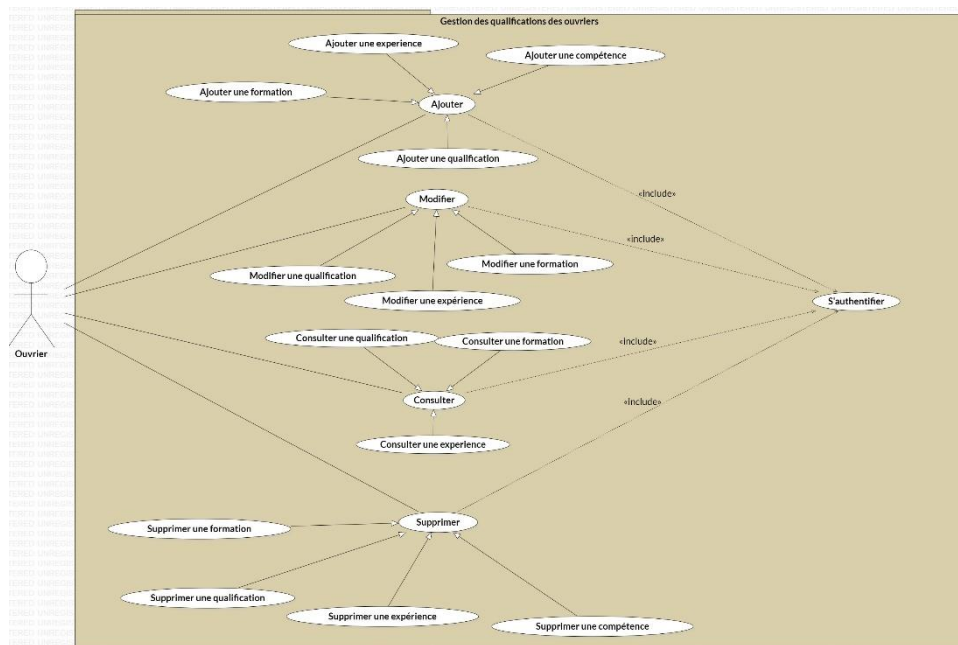


Figure 18 : diagramme de cas d'utilisation (gestion des qualifications des ouvriers)

5.1.4 Diagramme de cas d'utilisation de gestion des discussions

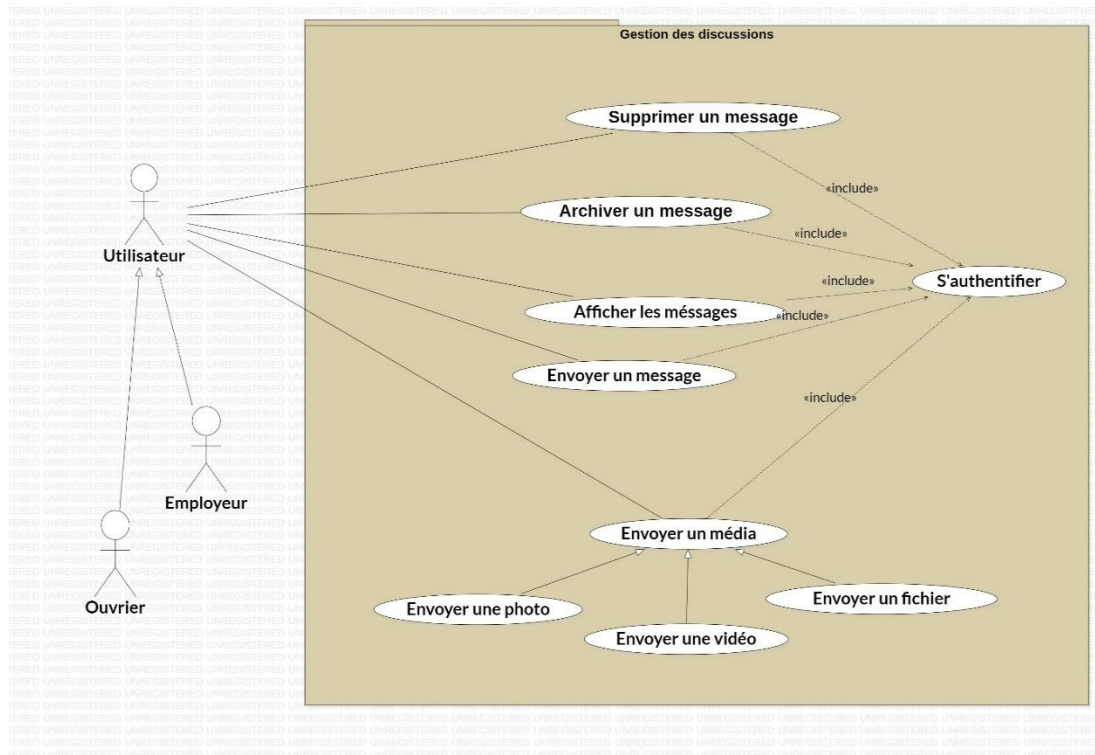


Figure 19 : diagramme de cas d'utilisation (gestion des discussions)

5.1.5 Diagramme de cas d'utilisation de gestion des comptes utilisateurs

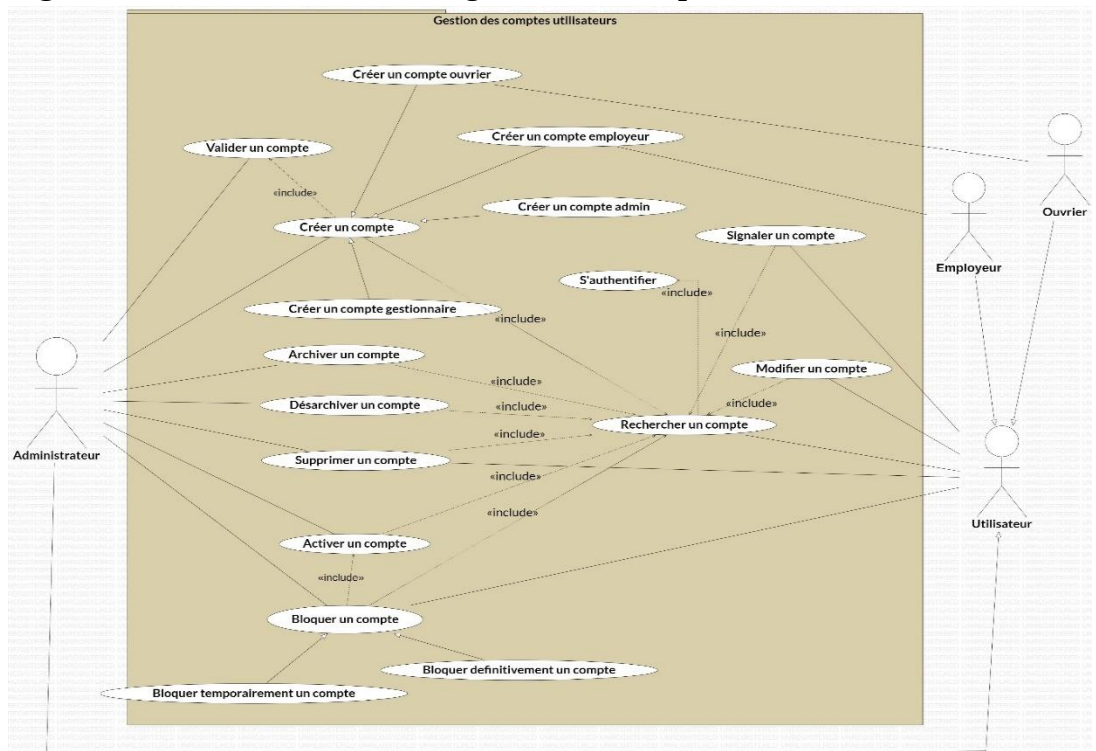


Figure 20 : diagramme de cas d'utilisation (gestion des comptes utilisateur)

5.2 Développement du modèle statique

Cette étape consistera à visualiser les principales entités du système à l'aide de diagrammes de classes, en fonction des packages définis lors de la phase d'analyse. Cela nous permettra de déduire un diagramme de classe générique pour le système.

Le diagramme de classe est un élément central de la modélisation orientée objet. Contrairement au diagramme de cas d'utilisation qui montre le système du point de vue des acteurs, le diagramme de classe en montre la structure interne. Il représente principalement les classes, qui contiennent les attributs et les opérations. Bien que le diagramme de classe fournisse une description statique du système, il ne spécifie pas comment utiliser les opérations. L'aspect dynamique de la modélisation est apporté par les diagrammes de séquences et d'activités.

Avant d'entrer dans le cœur de ce sous-chapitre, nous allons introduire quelques concepts de base nécessaires pour la réalisation de la vue structurelle d'un modèle UML :

- **Classe :** Une classe est une représentation d'un ensemble d'objets qui ont des caractéristiques et des actions similaires. Elle contient des attributs (ou propriétés) qui définissent les données qu'elle contient et des méthodes (ou opérations) qui définissent les actions qu'elle peut effectuer. Graphiquement, une classe est représentée par un rectangle avec son nom, ses attributs et ses méthodes.
- **Encapsulation :** L'encapsulation permet de définir les droits d'accès à une classe ou à ses propriétés. UML définit quatre niveaux d'encapsulation d'une propriété d'une classe ou d'une classe (public, private, protected, package).
- **Relation entre classes :** Elles permettent de modéliser les interactions et les dépendances entre les objets d'un système. Les classes sont liées entre elles par différentes relations qui expriment des associations, des agrégations, des compositions, des généralisations, des réalisations etc.
- **Héritage (Généralisation / Spécialisation) :** permet à une classe de dériver ou hériter les caractéristiques (attributs et méthodes) d'une autre classe. La classe qui hérite et appelés sous-classe ou classe dérivée, et la classe d'origine est appelée super-classe ou classe de base. Cela permet de réutiliser du code existant et de créer des classes plus spécifiques à partir de classes plus générales.
- **Le polymorphisme :** est un concept qui permet à une méthode d'être utilisée par des objets de différentes classes. Cela signifie que les objets peuvent avoir des structures et des comportements différentes classes. Le polymorphisme est généralement représenté par une flèche pointant vers une interface ou une classe abstraite dans un diagramme de classe, ce qui indique que les classes qui implémentent cette interface ou héritent de cette classe abstraite peuvent utiliser la méthode de manière polymorphe.

5.3 Diagramme de classe du système

Maintenant que nous avons vu les concepts de base des diagrammes de classes en UML, nous pouvons passer à la représentation concrète de notre système à travers les diagrammes de classes. Ces diagrammes permettent de visualiser toutes les classes qui composent le système ainsi que les relations et les héritages entre ces classes. Ils constituent un outil essentiel pour la compréhension de l'architecture, de la structure du système et facilite la communication entre les différents acteurs impliqués dans le projet.

5.3.1 Diagramme de classe du référentiel du domaine BPT

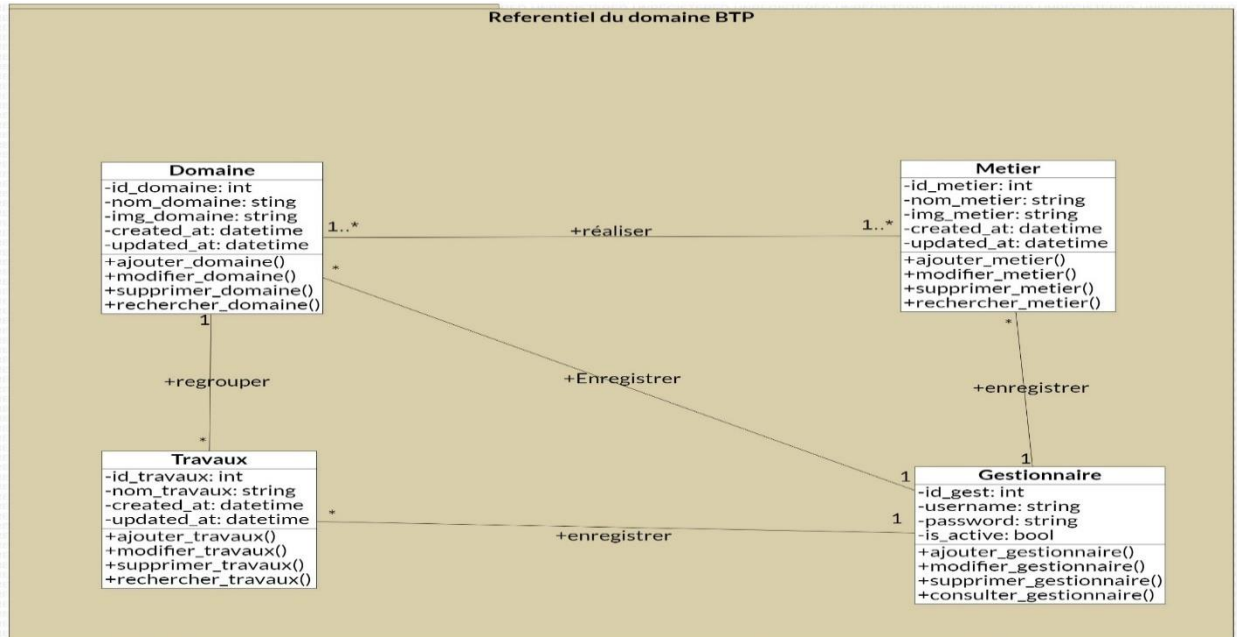


Figure 21 : Diagramme de classe du référentiel du domaine BPT

5.3.2 Diagramme de classe de gestion des offres et réservations

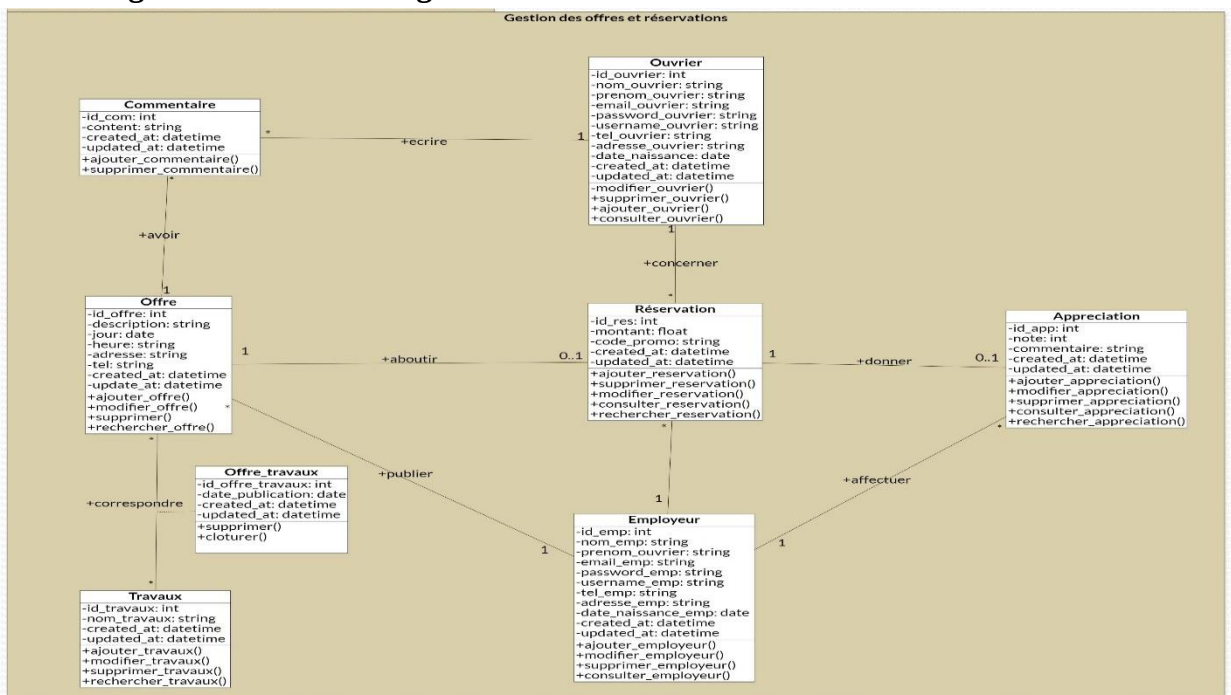


Figure 22 : Diagramme de classe (gestion des offres et réservations)

5.3.3 Diagramme de classe de gestion des qualifications des ouvriers

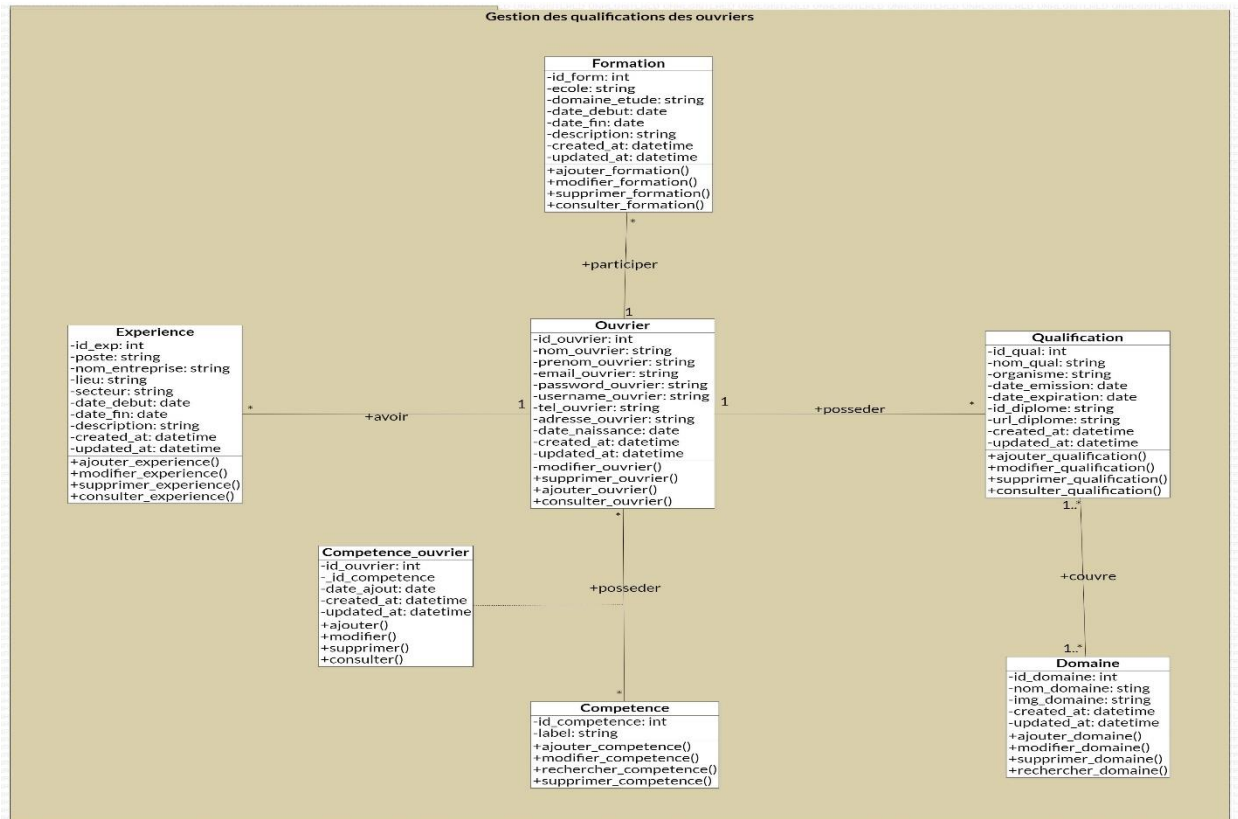


Figure 23 : diagramme de classe (gestion des qualifications des ouvriers)

5.3.4 Diagramme de classe de gestion des comptes utilisateurs

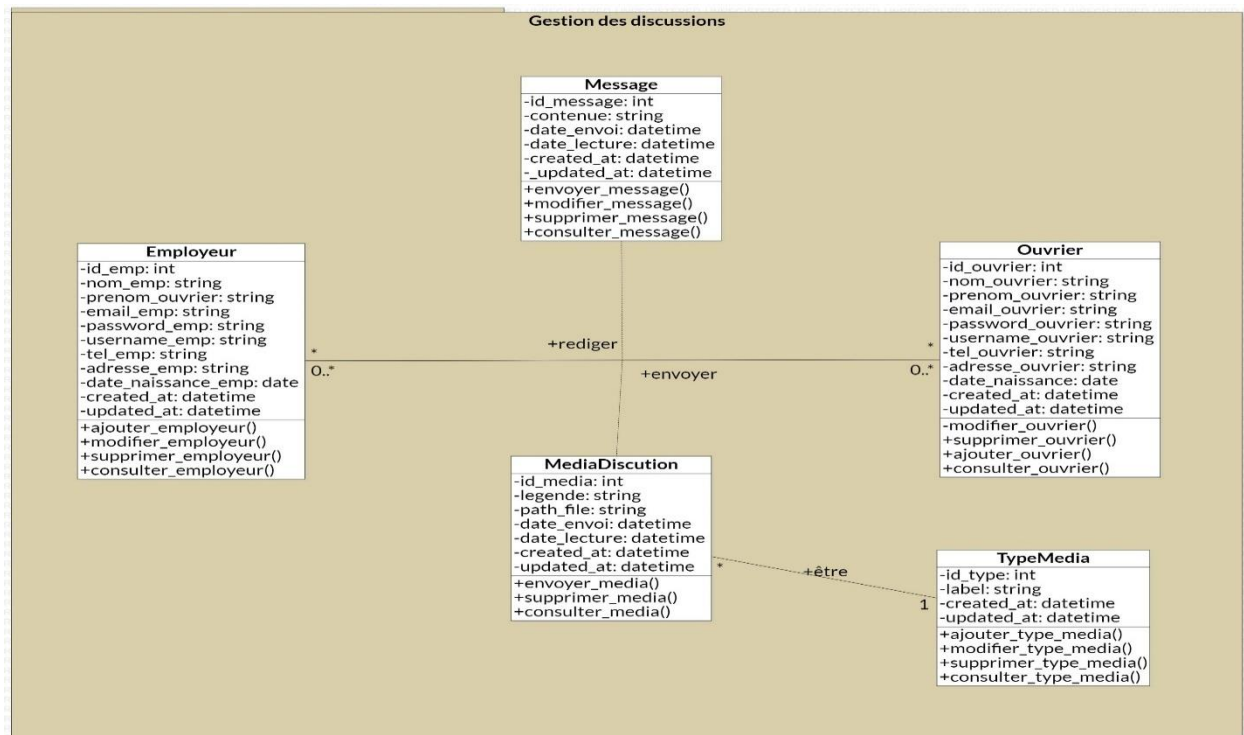


Figure 24 : diagramme de classe (gestion des comptes utilisateurs)

5.3.5 Diagramme de classe global du système

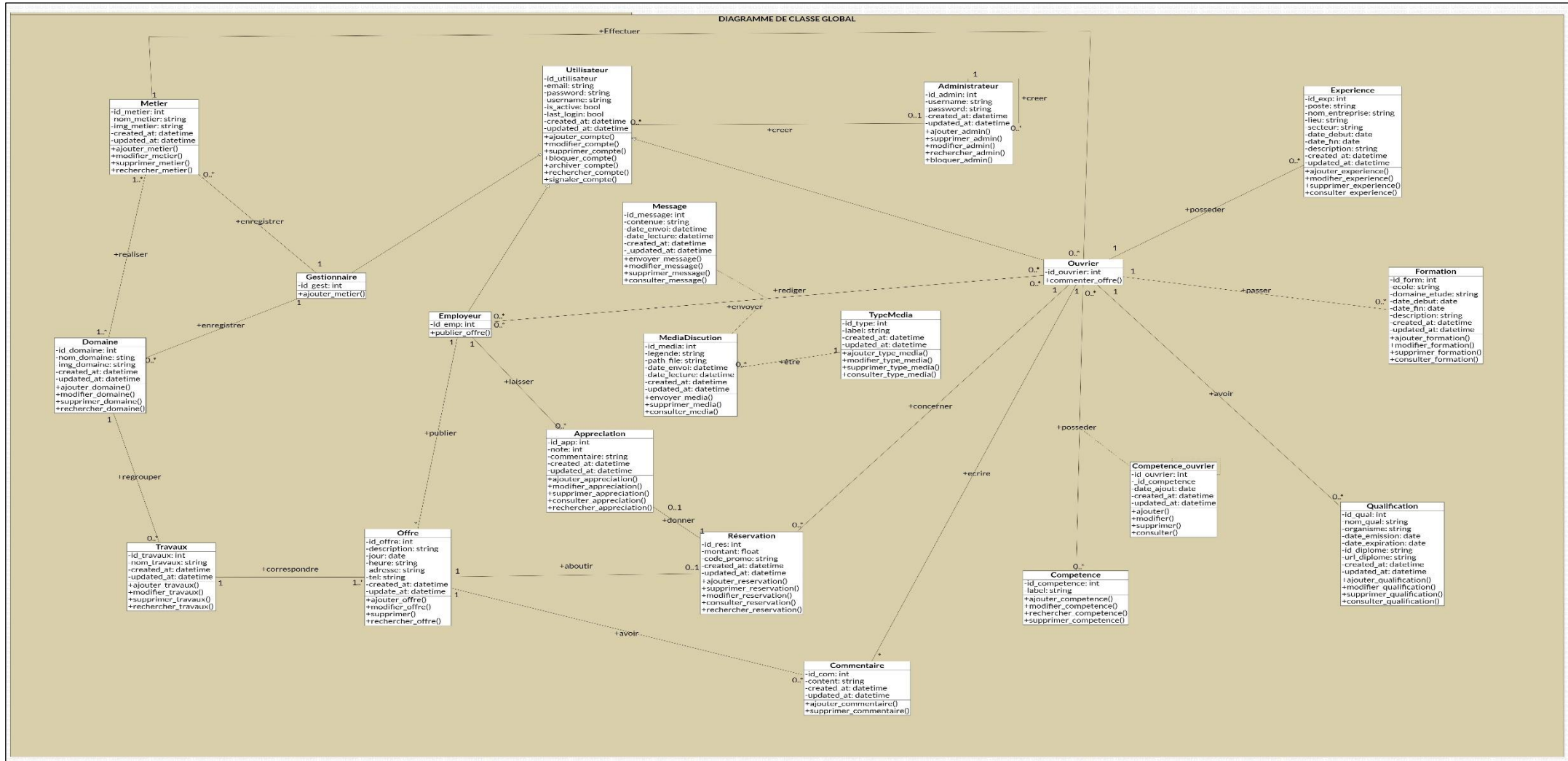


Figure 25 : diagramme de classe global du système

5.4 Conception architecturale

La conception architecturale consiste à définir l'architecture logicielle d'un système. Cette étape de la conception permet de définir l'organisation des différentes parties du système, ainsi que les interactions entre ces parties.

Cette partie est divisée en deux sections qui offrent deux perspectives conceptuelles complémentaires sur l'architecture du système. La première section fournit une vue d'ensemble, tandis que la seconde fournit une vue détaillée utilisant le langage de modélisation UML.

5.4.1 Architecture logicielle

L'architecture client-serveur est un modèle d'architecture informatique où les tâches sont réparties entre des serveurs et des clients. Dans cette architecture, les clients demandent des ressources ou des services, et les serveurs y répondent en fournissant ces ressources ou services. Les serveurs peuvent être dédiés à une tâche spécifique, comme la gestion des bases de données, la gestion des fichiers etc.

Le client est la partie de l'application qui est installée sur l'appareil de l'utilisateur et qui envoie des requêtes à un serveur distant pour obtenir des informations ou effectuer des actions. Le client peut être une application mobile, un navigateur web, un logiciel de bureau ou tout autre type de programme qui interagit avec le serveur.

Le serveur est un programme informatique qui fournit des services ou des ressources à des clients connectés à lui via un réseau. Le serveur reçoit des demandes de services de la part des clients, traite ces demandes et envoie les résultats correspondants aux clients.

L'architecture N-tiers est une architecture de développement logiciel qui permet de séparer les différentes fonctionnalités d'une application en plusieurs couches distinctes. Chaque couche est responsable d'une tâche spécifique et ne communique qu'avec les couches adjacentes.

L'architecture 1-tier est l'architecture la plus simple et la plus directe des architectures logicielles. Elle se caractérise par l'exécution de l'application sur une seule machine, généralement l'ordinateur personnel de l'utilisateur. Toutes les couches logicielles nécessaires au fonctionnement de l'application, à savoir la couche de présentation, la couche logique métier et la couche de données, sont regroupées sur une seule machine. Cette architecture est simple à mettre en œuvre, mais elle a ses limites en termes de connectivités et d'interopérabilité.

L'architecture 2-Tier est encore appelé architecture client-serveur simplifiée. Elle est couramment utilisée pour les applications Web où la couche de présentation est exécutée côté client et la couche de données est stockée et exécutée côté serveur. Contrairement à l'architecture 1-Tiers, cette architecture offre une meilleure connectivité et une meilleure, mais elle est limitée en termes de scalabilité et répartition de charge.

L'architecture 3-Tier :

L'architecture 3-Tier est une architecture logicielle est divisée en trois couches distinctes : la couche de présentation, la couche de logique métier et la couche de

données. Chacune de ces couches peut être exécutée sur un serveur différent, avec une communication à travers un réseau. La couche de présentation gère l'interface utilisateur, la couche de logique métier gère les opérations et la couche de données stocke les informations. Cette architecture offre une meilleure séparation des responsabilités et facilite ainsi la maintenance et le développement de l'application

Pour notre application, nous avons choisi de mettre en place une architecture 3-Tiers adaptée à nos besoins.

Notion de patron de conception

Un patron de conception est une solution éprouvée et générique à un problème de conception courant en informatique. Il s'agit d'une approche prédéfinie pour résoudre un type de problème de conception logicielle, qui peut être adaptée et appliquée à différents projets. Les patrons de conception sont considérés comme des bonnes pratiques de conception car ils ont été testés et utilisés dans de nombreux projets pour résoudre efficacement des problèmes similaires.

Pour notre application nous avons utilisé le pattern MVC.

Le pattern MVC (Model – View - Controller) est un patron de conception utilisé dans le développement logiciel pour séparer les données et la logique de présentation de l'interface utilisateur. Il consiste en trois éléments principaux : le modèle, la vue et le contrôleur.

Le Modèle (Model) contient les données de l'application et il est responsable de la récupération des données brutes depuis la base de données. Il organise et assemble ces données pour qu'elles soient prêtes à être traitées par le contrôleur, et peut inclure des requêtes SQL pour accéder à la base de données.

La vue (View) est responsable de l'interaction avec l'utilisateur et de l'affichage des données de l'application. Elle est chargée de présenter les informations au format approprié pour l'utilisateur, et de recueillir les actions de l'utilisateur pour les transmettre au contrôleur. Elle contient aucune logique métier ou d'accès aux données, mais utilise plutôt les données et les fonctions fournies par le contrôleur pour se mettre à jour en fonction des événements de l'application.

Le contrôleur (controller) est responsable de la gestion des actions de l'utilisateur. Il fait le lien entre la vue, qui représente l'interface utilisateur et le modèle, qui contient les données de l'application. Le contrôleur récupère les données du modèle, les traite et décide quelles informations doivent être affichées dans la vue.

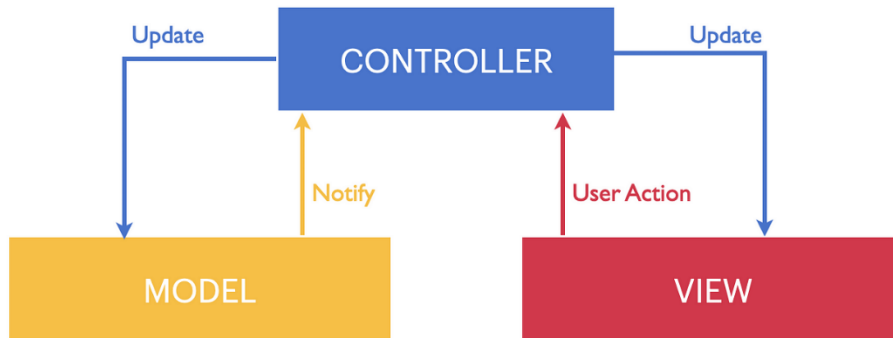


Figure 26 : architecture logicielle

5.5 Diagramme de composant

Le diagramme de composants illustre la structure organisationnelle du système en se concentrant sur les éléments logiciels tels que les modules (paquetages, fichiers sources, bibliothèques, exécutables), les données (fichiers, bases de données) ainsi que les éléments de configuration (paramètres, scripts, fichiers de commandes). Son objectif est de mettre en lumière les interrelations entre les composants, en identifiant clairement les dépendances entre eux et en décrivant les flux d'utilisation.

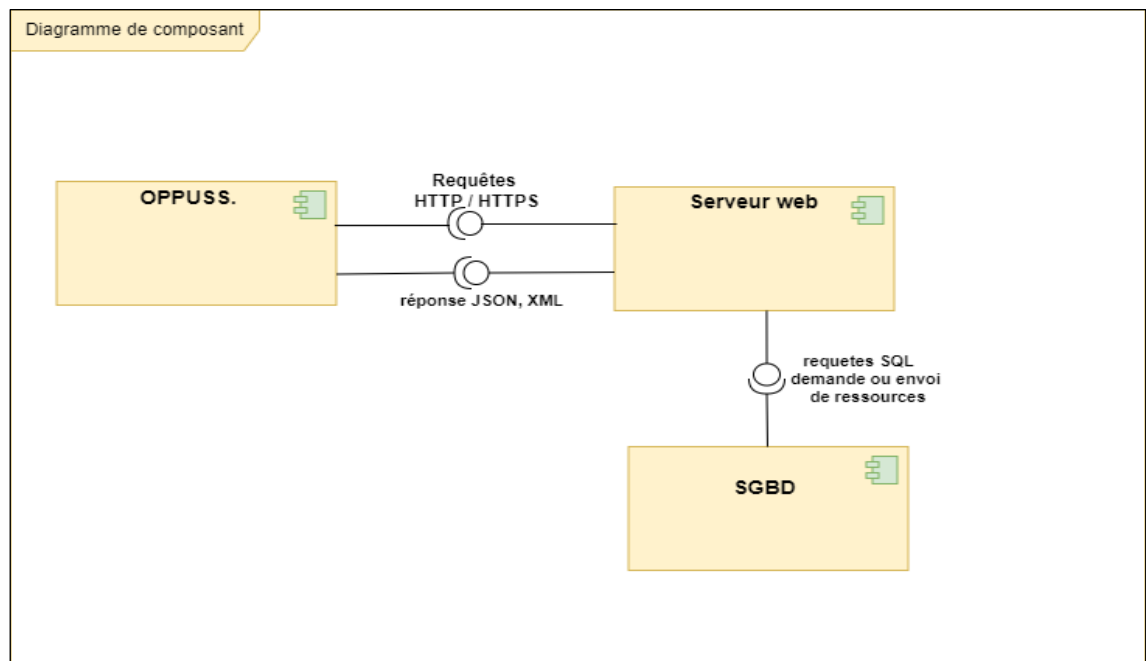


Figure 27 : diagramme de composant

Chapitre 6 : Implémentation

La phase d'implémentation permet de décrire l'environnement logiciel et matériel et les outils de production, les extraits de code sources et interface et l'environnement de mise en production de la solution.

6.1 Environnement technique de développement

L'Environnement technique de développement fait référence à l'ensemble des outils, des technologies, des langages de programmation, des plateformes logicielles, des méthodologies et d'autres ressources utilisées pour concevoir, développer, tester et déployer notre projet. Cette composante joue un rôle crucial dans la création et la réalisation d'une solution, car elle fournit le cadre et les moyens nécessaires pour mettre en œuvre les fonctionnalités et atteindre les objectifs du projet.

6.1.2 Environnement logiciel de production de la solution

- **Langage de programmations :**

- **Dart :**

Dart est un langage optimisé pour le client afin de développer des applications rapides sur n'importe quelle plateforme. Son objectif est d'offrir le langage de programmation le plus productif pour le développement multiplateforme, associé à une plate-forme d'exécution flexible pour les Framework d'applications.

Les langages sont définis par leur enveloppe technique les choix faits au cours du développement qui façonnent les capacités et les forces d'un langage. Dart est conçu pour une enveloppe technique particulièrement adaptée au développement client, en privilégiant à la fois le développement (rechargement à chaud avec état inférieur à la seconde) et les expériences de production de haute qualité sur une grande variété de cibles de compilation (Web, mobile et ordinateur de bureau).

Dart constitue également la base de Flutter. Dart fournit le langage et les runtimes qui alimentent les applications Flutter, mais Dart prend également en charge de nombreuses tâches de développement de base telle que le formatage, l'analyse et le test du code.

- **Python**

Est un langage de programmation interprété, multiparadigme et multiplateformes. Il favorise la programmation impérative structurée, fonctionnelle et orientée objet. Il est doté d'un typage dynamique fort, d'une gestion automatique du projet de fin d'étude par ramasse-miettes et d'un système de gestion d'exceptions ; il est ainsi similaire à Perl, Ruby, Scheme, Smalltalk et Tcl.

Le langage Python fonctionne sur la plupart des plates-formes informatiques, des smartphones aux ordinateurs centraux, de Windows à Unix avec notamment GNU/Linux en passant par mac OS, ou encore Android, iOS, et peut aussi être traduit en Java ou .NET. Il est conçu pour optimiser la

productivité des programmeurs en offrant des outils de haut niveau et une syntaxe simple à utiliser.

Il est également apprécié par certains pédagogues qui y trouvent un langage où la syntaxe, clairement séparée des mécanismes de bas niveau, permet une initiation aisée aux concepts de base de la programmation.

- **JSON :**

JSON est un format léger d'échange de données. Il est facile à lire ou à écrire pour des humains. Il est aisément analysable ou qui peut-être générer par des machines. Il est basé sur un sous-ensemble du langage de programmation JavaScript. JSON est un format texte complètement indépendant de tout langage, mais les conventions qu'il utilise seront familières à tout programmeur habitué aux langages descendant du C, comme par exemple : C lui-même, C++, C#, Java, JavaScript, Perl, Python et bien d'autres. Ces propriétés font de JSON un langage d'échange de données idéal.

- **Framework :**

- **Flutter :**

Flutter est un kit de développement logiciel (SDK) d'interface utilisateur open-source créé par Google. Il est utilisé pour développer des applications pour Android, iOS, Linux, Mac, Windows, Google Fuchsia et le web à partir d'une seule base de code.

- **Django :**

Django est un Framework de développement web open source en Python qui s'inspire du principe MVC ou MTV (la vue est gérée par un gabarit). Il a pour but de rendre le développement web 2.0 simple et rapide composé de trois parties distinctes :

Un langage de gabarits flexible qui permet de générer du HTML, XML ou tout autre format texte. Un contrôleur fourni sous la forme d'un « remapping » d'URL à base d'expressions rationnelles. Une API d'accès aux données est automatiquement générée par le cadre compatible CRUD. Inutile d'écrire des requêtes SQL associées à des formulaires, elles sont générées automatiquement par l'ORM.

En plus de l'API d'accès aux données, une interface d'administration fonctionnelle est générée depuis le modèle de données. Un système de validation des données entrées par l'utilisateur est également disponible et permet d'afficher des messages d'erreur automatiques.

Sont également inclus :

Un serveur web léger permettant de développer et tester ses applications en temps réel sans déploiement. Un système élaboré de traitement des formulaires muni de widgets permettant d'interagir entre du HTML et une base de données. De nombreuses possibilités de contrôles et de traitements sont fournies. Un cadre de cache web pouvant utiliser différentes méthodes (MemCached, système de fichier, base de données, personnalisé).

Le support de classes intermédiaires (intergiciel) qui peuvent être placées à des stades variés du traitement des requêtes pour intégrer des traitements employeurs (cache, internationalisation, accès...). Une prise en charge complète d'Unicode.

Django peut être considéré comme une boîte à outils où chaque module peut fonctionner de façon indépendante.

Nous avons utilisé :

- **AbstractUser :**

L'utilisateur de la classe AbstractUser permet de remplacer la classe User par défaut fournie par Django et de personnaliser le modèle d'utilisateur selon les besoins de l'application, en fournissant une base solide pour la gestion des utilisateurs.

- **API Rest :**

Une API Rest (Representational State Transfer) est une architecture logicielle utilisée pour concevoir des interfaces d'application qui permettent à différentes applications de communiquer entre elles.

- **MySQL :**

Est un système de gestion de bases de données relationnelles (SGBDR). Il est distribué sous une double licence GPL et propriétaire. Il fait partie des logiciels de gestion de base de données les plus utilisés au monde, autant par le grand public (applications web principalement) que par des professionnels.

Outils de conception :

- **StarUML :**

Est un logiciel de modélisation UML, qui a été « cédé comme open source » par son éditeur, à la fin de son exploitation commerciale (qui visiblement continue ...), sous une licence modifiée de GNU GPL. StarUML gère la plupart des diagrammes spécifiés dans la norme UML 2.0.

- **DrawIo :**

DrawIo est un logiciel de design graphique multiplateforme gratuit et open source développée en HTML5 et JavaScript. Son interface peut être utilisée pour créer des diagrammes tels que des organigrammes, des structures filaires, des diagrammes UML, des organigrammes et des diagrammes de réseau.

- **Power Design (AMC)**

Power Design est un outil de création de modèles de processus métier et de modèles de données conceptuels, logiques et physiques pour la conception de

bases de données, y compris des modèles relationnels et dimensionnels. Power Design peut coordonner le modèle de processus métier avec la conception de la base de données, en veillant à ce que les étapes de processus qui créent des données aient des représentations de données dans le modèle logique. PowerAMC peut créer la base de données réelle à partir du modèle physique et créer différentes implémentations physiques à partir d'un seul modèle logique. Power Design peut également procéder à un reverse engineering de bases de données existantes dans un diagramme de modèle. PowerAMC fonctionne avec de nombreux systèmes de gestion de base de données (SGBD). Les principaux résultats de l'outil incluent des diagrammes entité-relation (ER), des rapports d'analyse d'impact sur les modifications de conception et des rapports standards ou personnalisés sur tous les objets de la conception (tables, champs, relations).

Outils de développement :

- **SDK :**

Est un ensemble d'outils de développement logiciel fourni par un éditeur de logiciel ou un tiers pour faciliter la création d'applications pour une plateforme spécifique.

Outils de test :

- **AVD :**

L'Android Virtual Device est un émulateur de terminal sous Android, il permet de lancer sur la machine du développeur un terminal virtuel représentant à l'écran un téléphone embarquant Android. C'est bien évidemment un outil indispensable pour le développement mobile. A chaque version d'Android est associée une version de l'émulateur, permettant au développeur de voir exactement à quoi ressemblera son application sur un matériel réel.

- **Google Chrome :**

Est un navigateur web gratuit développé par Google. Il a été lancé en septembre 2008 pour Windows, et a ensuite été étendu à d'autres systèmes d'exploitation tels que MacOS, Linux, Android et iOS.

Outils supplémentaires :

- **Postman :**

Postman est une plate-forme API pour la création et l'utilisation d'API. Postman simplifie chaque étape du cycle de vie des API et rationalise la collaboration afin que vous puissiez créer de meilleures API plus rapidement.

- **Visual Studio Code :**

VS code est un éditeur de code source gratuit et open-source développé par Microsoft. Il est disponible pour Windows, Linux et MacOS.

▪ Git et GitHub :

Il existe de nombreux logiciels de gestion de versions, comme SVN (Subversion), Mercurial et Git. Pour notre projet, nous avons utilisé Git (prononcez « guite »). En effet, ils sont quasi exclusivement utilisés pour gérer des codes sources, car ils sont capables de suivre l'évolution d'un fichier texte ligne de code par ligne de code. Ces logiciels sont fortement conseillés pour gérer un projet informatique. Ces outils suivent l'évolution de vos fichiers source et gardent les anciennes versions de chacun d'eux. Et le dépôt est stocker sur GitHub.

Les webservices :

Un Webservice est une fonctionnalité qu'on peut appeler à distance à travers un réseau (comme un intranet d'entreprise ou internet lui-même).

Nous distinguons deux générations de Services Web :

- Webservice 1.0 basé sur le protocole SOAP.
- Webservice 2.0 basé sur le style architectural REST.

SOAP et REST sont deux styles d'architecture pour la conception de service web. La principale différence entre les deux sont le degré de couplage entre le client et le serveur.

Avec SOAP, le client est étroitement lié au serveur, ce qui signifie que tout changement apporté à l'un ou l'autre peut affecter l'ensemble du système. Des mises à jour doivent être effectuées des deux côtés si nécessaire.

En revanche, REST est plus flexible et moins couplé, grâce à l'utilisation de protocoles et de méthodes standardisées. Le client peut utiliser les méthodes standardisées pour le type de média dont il a besoin, sans avoir besoin d'une connaissance approfondie de l'API. REST est également indépendant de tout protocole particulier, ce qui signifie que tout protocole disposant d'un schéma standard pour une URI peut être utilisé.

Pour ces raisons, nous avons opté pour l'utilisation de webservices REST dans notre projet. Nous avons créé nos propres webservices REST et utilisé également des webservices REST existants en utilisant des messages JSON pour communiquer entre le client et le serveur.

6.1.2 Environnement matériel de production de la solution

Les matériels utilisés pour développer cette solution numérique sont décrits dans le tableau ci-dessous.

Marque	Modèle	Processeur	Projet de fin d'étude RAM	Projet de fin d'étude ROM	Système d'exploitation
HP	ZBook	Intel(R)Core (TM) I7 CPU 2.70GHz	32 GB	500 SSD	Windows 11
HP	EliteBook 8470p	Intel(R) Core (TM) I7 CPU 2.70GHz	16 GB	1 TB SSD	Kali Linux

Tableau 5 : Environnement matériel

6.2 Diagramme de déploiement

Le diagramme de déploiement est un outil visuel statique permet de visualiser l'infrastructure physique utilisée pour exploiter le système ainsi que la répartition des différents composants et leurs interrelations.

Dans le cadre de ce projet, nous avons utilisé un ensemble de ressources matérielles, notamment un smartphone Android, un navigateur client, ainsi que nos propres API REST.

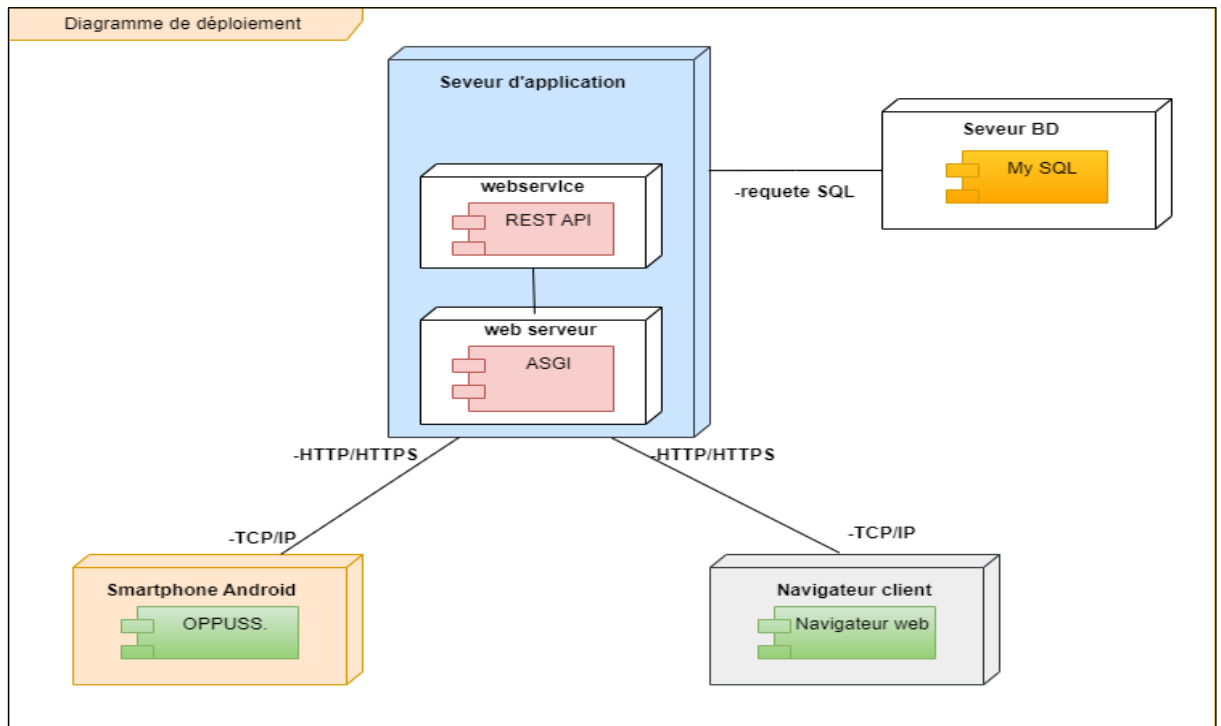


Figure 28 : diagramme de déploiement

6.3 Production du logiciel

La production du logiciel se réfère au processus de création, de développement, de tests et de mise en œuvre effective du logiciel ou de l'application. Cette phase est cruciale pour transformer la conception théorique en une solution concrète et fonctionnelle.

6.3.1 Schéma relationnel

Le modèle relationnel est une approche structurée pour représenter et organiser les données dans une base de données. Il permet de modéliser les relations entre différents ensembles d'informations, en créant une structure de tableaux interconnectés. Chaque table représente une entité spécifique et chaque colonne représente une caractéristique particulière de cette entité. Le modèle relationnel permet donc de définir des contraintes et des règles pour les données dans une base de données, ce qui garantit l'intégrité des informations et facilite leurs accès et leur utilisation.

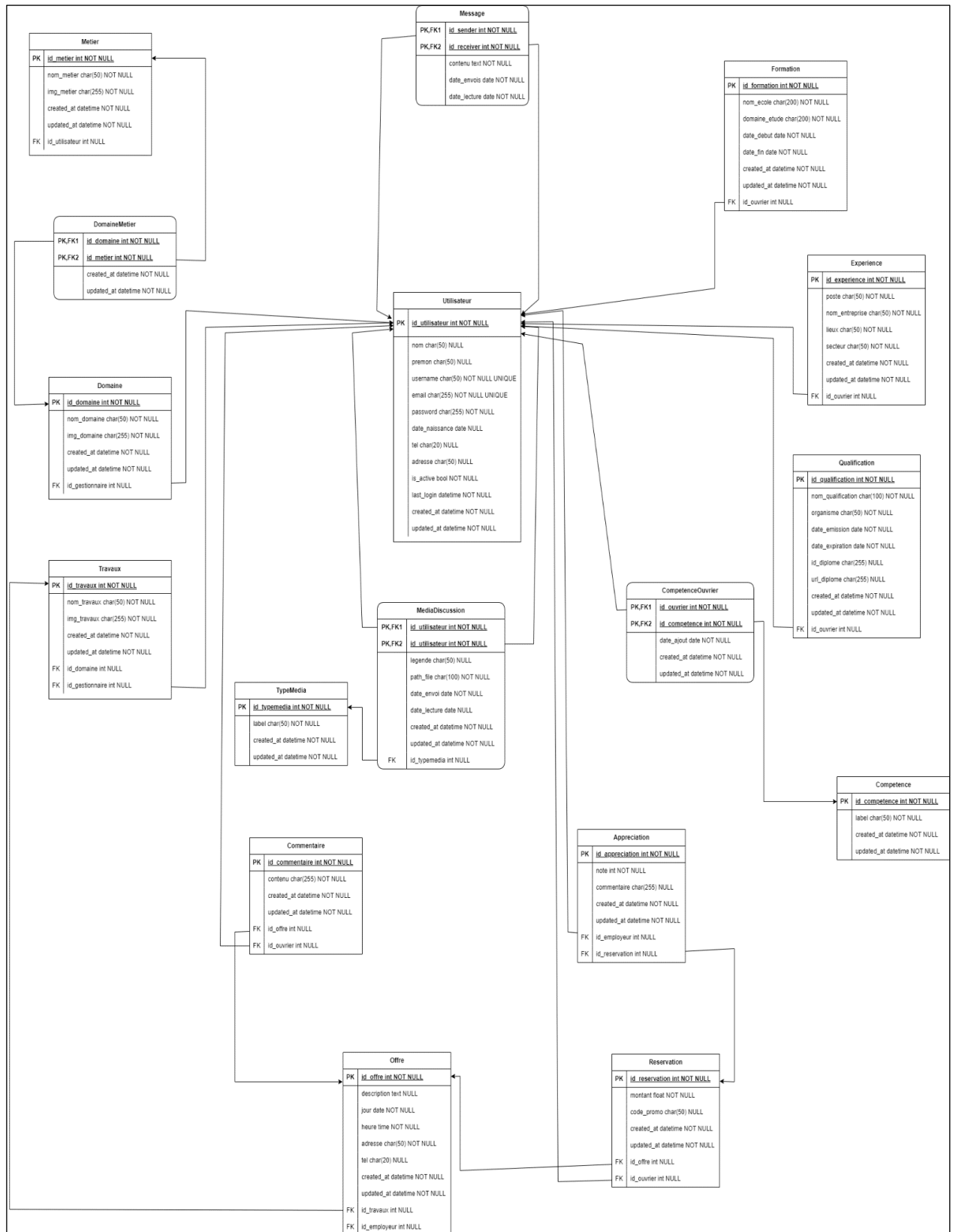


Figure 29: schéma relationnel

6.3.2 Modèle physique des données

Le modèle physique de données est la réalisation pratique et concrète du modèle logique des données, grâce à l'utilisation d'un logiciel de gestion de bases de données. Ce processus implique la création des structures stockage nécessaires pour les données, en fonction de leur type et de leur taille définis dans le modèle logique. Ainsi, le modèle physique des données permet de traduire les concepts abstraits du modèle logique en des éléments concrets de stockage, tels que des fichiers ou des tables, qui peuvent être utilisés pour stocker et manipuler les données. En somme, le modèle physique des données constitue la mise en œuvre technique du modèle logique des données, et il est essentiel pour garantir la qualité et la performance de la base de données.

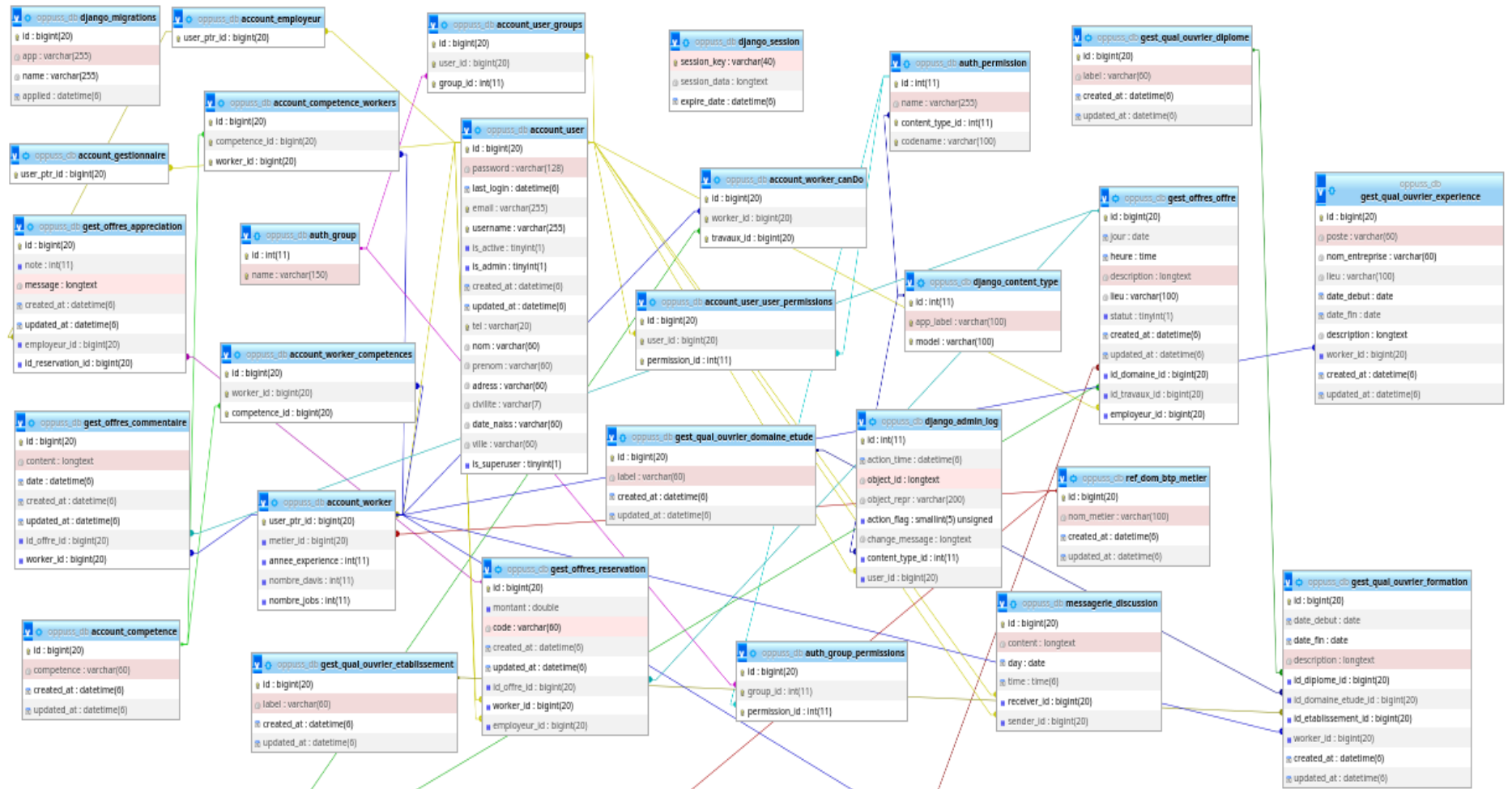


Figure 30 : Modèle physique

6.3.3 Code source de l'application

```

1  import 'package:flutter/material.dart';
2  import 'package:go_router/go_router.dart';
3  import 'package:oppuss/api/auth_provider.dart';
4  import 'package:oppuss/modules/chat_page.dart';
5  import 'package:oppuss/utils/theme.dart';
6  import 'package:oppuss/views/auth/login_screen.dart';
7  import 'package:oppuss/views/auth/sign_up_ouvrier.dart';
8  import 'package:oppuss/views/fullscreen.dart';
9  import 'package:oppuss/views/home/home_screen.dart';
10 import 'package:oppuss/views/particulars/coments.dart';
11 import 'package:oppuss/views/particulars/help.dart';
12 import 'package:oppuss/views/particulars/notification_view.dart';
13 import 'package:oppuss/views/particulars/offer_detail.dart';
14 import 'package:oppuss/views/particulars/offer_more_detail.dart';
15 import 'package:oppuss/views/particulars/offer_update.dart';
16 import 'package:oppuss/views/particulars/profile_edit_view.dart';
17 import 'package:flutter/services.dart';
18 import 'package:oppuss/views/particulars/publications.dart';
19 import 'package:oppuss/views/particulars/safe.dart';
20 import 'package:oppuss/views/particulars/settings.dart';
21 import 'package:oppuss/views/particulars/about.dart';
22 import 'package:oppuss/views/particulars/worker_profile.dart';
23 import 'package:oppuss/views/splash_screen.dart';
24 import 'package:oppuss/views/welcome_screen.dart';
25 import 'package:provider/provider.dart';
26 import 'views/auth/sign_up_particuler.dart';
27
28 void main() {
29
30   SystemChrome.setSystemUIOverlayStyle(
31     const SystemUIOverlayStyle(statusBarColor: Colors.transparent));
32
33   runApp(MyApp());
34 }
35
36 class MyApp extends StatelessWidget {
37   MyApp({super.key});
38
39   // This widget is the root of your application.
40   @override
41   Widget build(BuildContext context) {
42     return MultiProvider(
43       providers: [
44         ChangeNotifierProvider(create: (_) => AuthProvider()),
45       ],
46       child: MaterialApp.router(
47         routerConfig: _router,
48         debugShowCheckedModeBanner: false,
49       ),
50     );
51   }
52
53   final GoRouter _router = GoRouter(
54     routes: [
55       GoRoute(path: "/", builder: (context, state) => const SplashScreen(),
56         routes: [
57           GoRoute(
58             path: "worker",
59             builder: (context, state) => const SignUpScreenOuvrier(),
60           ),
61         ],
62       ),
63       GoRoute(
64         path: "/home",
65         builder: (context, state) => const HomeScreen(),
66         routes: [
67           GoRoute( path: "profile", builder: (context, state) => const EditProfilePage(),),
68           GoRoute( path: "add_offer", builder: (context, state) => const AddOffer(),),
69           GoRoute( path: "notification", builder: (context, state) => const NotificationView(), ),
70           GoRoute( path: "message", builder: (context, state) => ChatPage(),),
71           GoRoute( path: "offer_detail", builder: (context, state) => const OfferDetailView(),
72             routes: [
73               GoRoute(path: "coments", builder: (context, state) => const ComentView(),),
74               GoRoute( path: "worker_profile", builder: (context, state) => const WorkerProfile(),),
75               GoRoute(path: "update_offer", builder: (context, state) => const UpdateOfferPage() ),
76               GoRoute(path: "view", builder: (context, state) => const OfferMoreDetailPage())
77             ],
78           ),
79           GoRoute( path: "settings", builder: (context, state) => const SettingsView(),),
80           GoRoute( path: "help", builder: (context, state) => const HelpView(),),
81           GoRoute( path: "safe", builder: (context, state) => const SafeView(),),
82           GoRoute( path: "worker", builder: (context, state) => const SignUpScreenOuvrier(),),
83           GoRoute( path: "about", builder: (context, state) => const AboutView(),),
84           GoRoute( path: "worker_profile", builder: (context, state) => const WorkerProfile(),),
85
86           //Route pour L'authentification
87           GoRoute(path: "login", builder: (context, state) => const LoginScreen(),),
88           GoRoute(path: "user_register", builder: (context, state) => const SignUpScreenParticuler(),),
89         ],
90       ),
91       GoRoute(path: "/welcome", builder: (context, state) => const WelcomeAuth(),),
92       GoRoute(path: "/loading_screen", builder: (context, state) => LoadingScreen())
93     ],
94   );
95 }

```



```

1  import 'package:flutter/material.dart';
2  import 'package:go_router/go_router.dart';
3  import 'package:http/http.dart' as http;
4  import 'dart:convert';
5  import 'package:shared_preferences/shared_preferences.dart';
6
7  // ignore: constant_identifier_names
8  const String api_worker_profile_view = "http://10.0.2.2:8000/api/user/profile";
9  const String api_login_view = "http://10.0.2.2:8000/api/user/login/";
10 const String api_get_current_user = "http://10.0.2.2:8000/api/user/get_current_user/";
11 const String api_domaine_view = "http://10.0.2.2:8000/api/ref-btp/domaines/";
12 const String api_travaux_view = "http://10.0.2.2:8000/api/ref-btp/travaux/";
13 const String api_metier_view = "http://10.0.2.2:8000/api/ref-btp/metier/";
14
15
16 Future<void> fetchDomaines() async {
17
18     final response = await http.get(Uri.parse(api_domaine_view));
19
20     final decodedResponse = utf8.decode(response.bodyBytes);
21
22     if (response.statusCode == 200) {
23         // Si la requête a réussi, on enregistre dans les domaines dans le fichier de préférence de l'app
24         SharedPreferences prefs = await SharedPreferences.getInstance();
25
26         prefs.setString('domaines', decodedResponse);
27
28     } else {
29         // Sinon, on lance une exception avec le message d'erreur renvoyé par l'API Django
30         throw Exception('Erreur lors de la récupération des domaines: ${response.reasonPhrase}');
31     }
32 }
33
34 Future<void> fetchTravaux() async {
35
36     final response = await http.get(Uri.parse(api_travaux_view));
37
38     final decodedResponse = utf8.decode(response.bodyBytes);
39
40     if (response.statusCode == 200) {
41         // Si la requête a réussi, on enregistre dans les domaines dans le fichier de préférence de l'app
42         SharedPreferences prefs = await SharedPreferences.getInstance();
43
44         prefs.setString('travaux', decodedResponse);
45
46     } else {
47         // Sinon, on lance une exception avec le message d'erreur renvoyé par l'API Django
48         throw Exception('Erreur lors de la récupération des travaux: ${response.reasonPhrase}');
49     }
50 }
51
52 Future<void> fetchMetier() async {
53
54     final response = await http.get(Uri.parse(api_metier_view));
55
56     final decodedResponse = utf8.decode(response.bodyBytes);
57
58     if (response.statusCode == 200) {
59         // Si la requête a réussi, on enregistre dans les domaines dans le fichier de préférence de l'app
60         SharedPreferences prefs = await SharedPreferences.getInstance();
61
62         prefs.setString('metiers', decodedResponse);
63
64     } else {
65         // Sinon, on lance une exception avec le message d'erreur renvoyé par l'API Django
66         throw Exception('Erreur lors de la récupération des metiers: ${response.reasonPhrase}');
67     }
68 }
69
70
71

```

```

1  import 'package:flutter/material.dart';
2  import 'package:go_router/go_router.dart';
3  import 'package:http/http.dart' as http;
4  import 'dart:convert';
5  import 'package:shared_preferences/shared_preferences.dart';
6
7  // ignore: constant_identifier_names
8  const String api_worker_profile_view = "http://10.0.2.2:8000/api/user/profile";
9  const String api_login_view = "http://10.0.2.2:8000/api/user/login/";
10 const String api_get_current_user = "http://10.0.2.2:8000/api/user/get_current_user/";
11 const String api_domaine_view = "http://10.0.2.2:8000/api/ref-btp/domaines/";
12 const String api_travaux_view = "http://10.0.2.2:8000/api/ref-btp/travaux/";
13 const String api_metier_view = "http://10.0.2.2:8000/api/ref-btp/metier/";
14
15
16 Future<void> fetchDomaines() async {
17
18     final response = await http.get(Uri.parse(api_domaine_view));
19
20     final decodedResponse = utf8.decode(response.bodyBytes);
21
22     if (response.statusCode == 200) {
23         // Si la requête a réussi, on enregistre dans les domaines dans le fichier de préférence de l'app
24         SharedPreferences prefs = await SharedPreferences.getInstance();
25
26         prefs.setString('domaines', decodedResponse);
27     } else {
28         // Sinon, on lance une exception avec le message d'erreur renvoyé par l'API Django
29         throw Exception('Erreur lors de la récupération des domaines: ${response.reasonPhrase}');
30     }
31 }
32
33
34 Future<void> fetchTravaux() async {
35
36     final response = await http.get(Uri.parse(api_travaux_view));
37
38     final decodedResponse = utf8.decode(response.bodyBytes);
39
40     if (response.statusCode == 200) {
41         // Si la requête a réussi, on enregistre dans les domaines dans le fichier de préférence de l'app
42         SharedPreferences prefs = await SharedPreferences.getInstance();
43
44         prefs.setString('travaux', decodedResponse);
45     } else {
46         // Sinon, on lance une exception avec le message d'erreur renvoyé par l'API Django
47         throw Exception('Erreur lors de la récupération des travaux: ${response.reasonPhrase}');
48     }
49 }
50
51
52 Future<void> fetchMetier() async {
53
54     final response = await http.get(Uri.parse(api_metier_view));
55
56     final decodedResponse = utf8.decode(response.bodyBytes);
57
58     if (response.statusCode == 200) {
59         // Si la requête a réussi, on enregistre dans les domaines dans le fichier de préférence de l'app
60         SharedPreferences prefs = await SharedPreferences.getInstance();
61
62         prefs.setString('metiers', decodedResponse);
63     } else {
64         // Sinon, on lance une exception avec le message d'erreur renvoyé par l'API Django
65         throw Exception('Erreur lors de la récupération des metiers: ${response.reasonPhrase}');
66     }
67 }
68
69
70
71

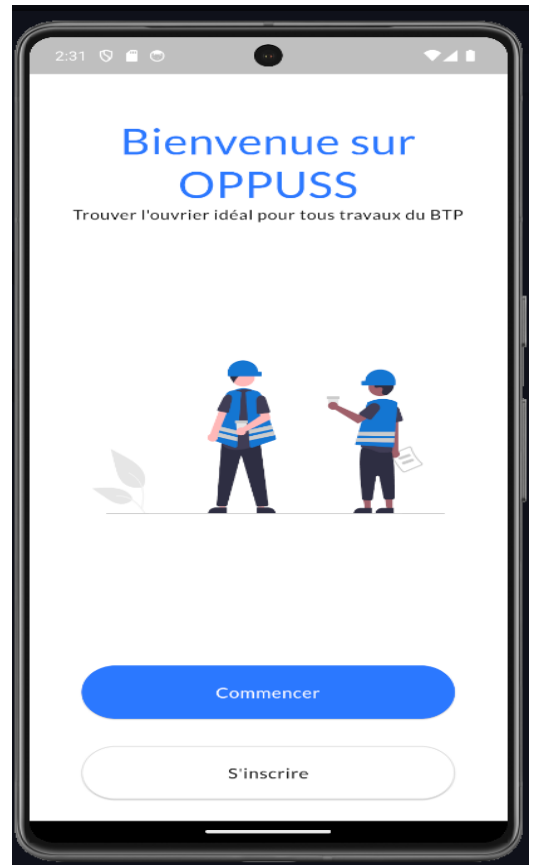
```

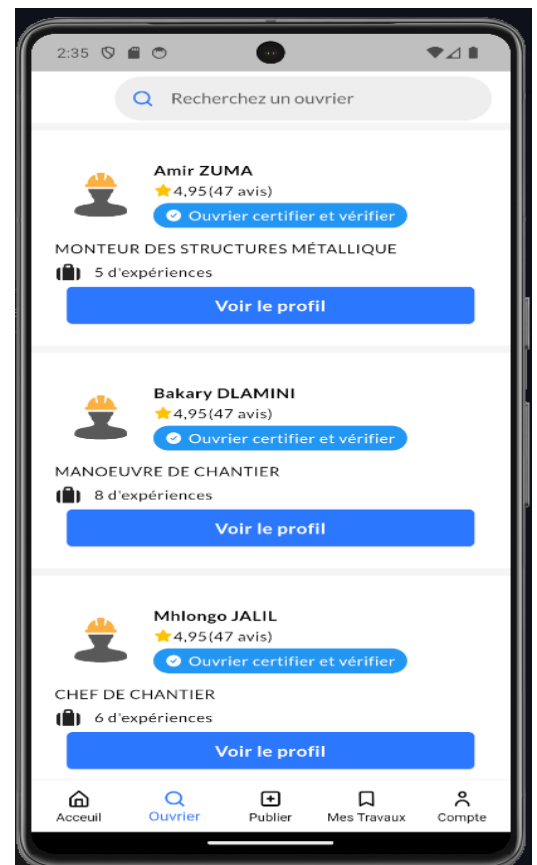
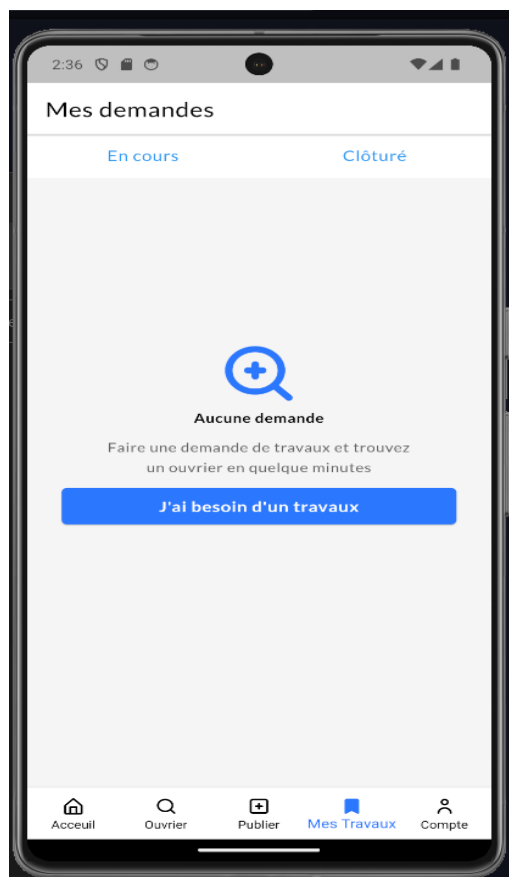
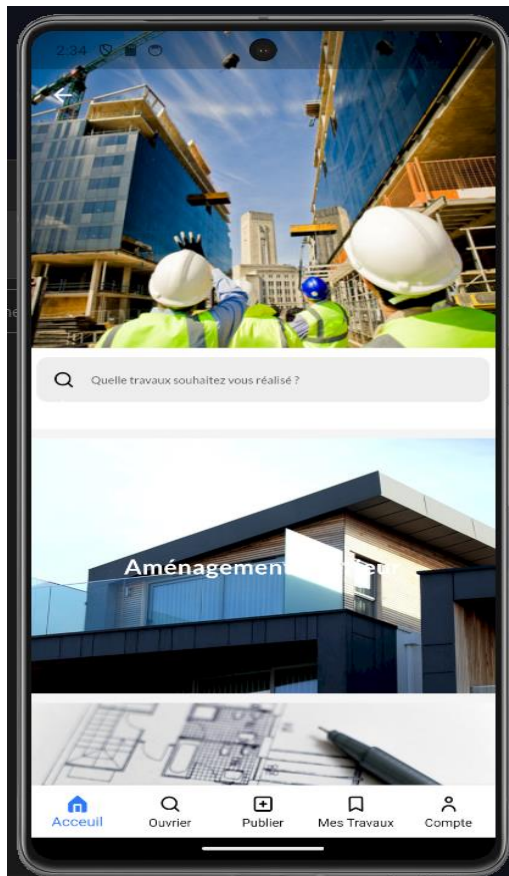
```

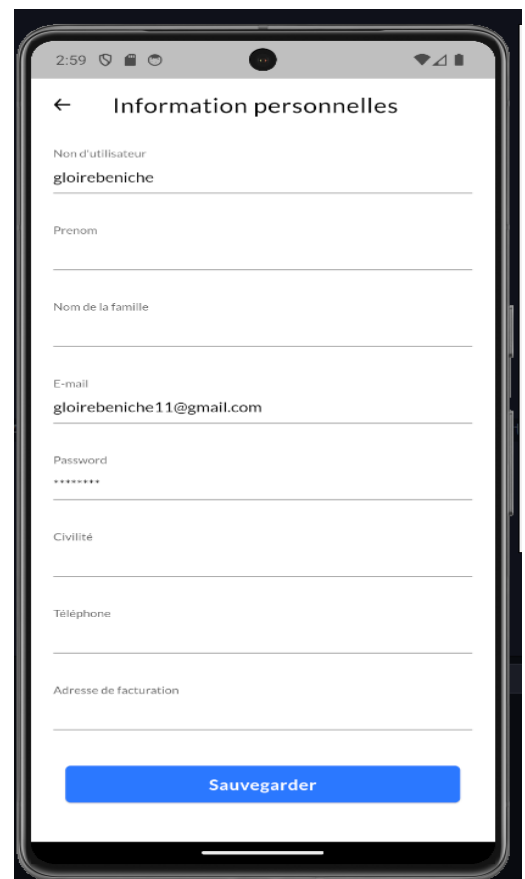
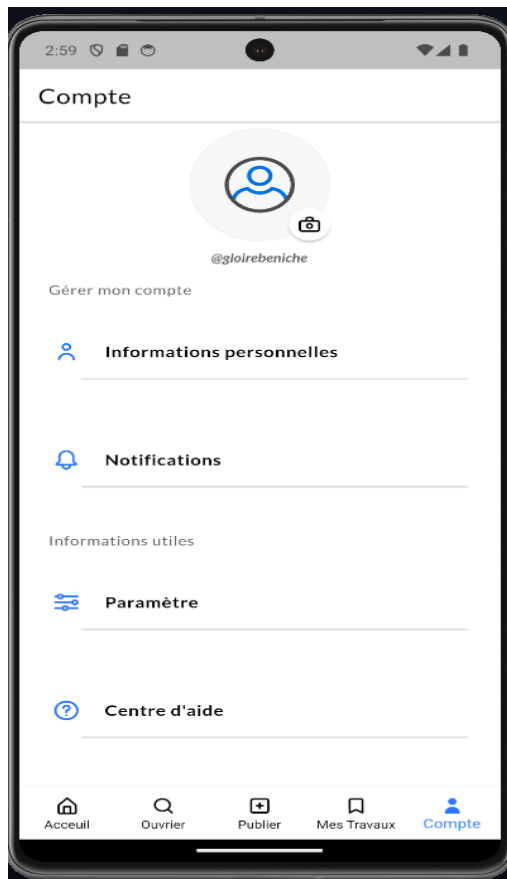
1  import 'package:flutter/material.dart';
2  import 'package:get/get.dart';
3  import 'package:intl/date_symbol_data_local.dart';
4  import 'package:oppuss/views/splash_screen.dart';
5  import 'package:provider/provider.dart';
6  import 'api/auth_provider.dart';
7
8  void main() async {
9    Get.testMode = true;
10   // Ajoutez L'appel à initializeDateFormatting avec La locale appropriée
11   await initializeDateFormatting('fr_FR', null);
12   runApp(
13     MultiProvider(
14       providers: [
15         ChangeNotifierProvider(create: (_) => AuthProvider()),
16       ],
17       child: GetMaterialApp(
18         debugShowCheckedModeBanner: false,
19         theme: ThemeData.from(
20           colorScheme: const ColorScheme.light(),
21         ).copyWith(
22           pageTransitionsTheme: const PageTransitionsTheme(
23             builders: <TargetPlatform, PageTransitionsBuilder>{
24               TargetPlatform.android: ZoomPageTransitionsBuilder(),
25             },
26           ),
27         ),
28         home: const MyApp(),
29       ),
30     ),
31   );
32 }
33
34 class MyApp extends StatelessWidget {
35   const MyApp({Key? key}) : super(key: key);
36
37   @override
38   Widget build(BuildContext context) {
39     return const MaterialApp(
40       debugShowCheckedModeBanner: false,
41       home: SplashScreen(),
42     );
43   }
44 }
45

```

6.3.4 Extraits des écrans de l'application







Conclusion

En conclusion, nous avons présenté les différents aspects des plateformes de mise à disposition des ouvriers dans le domaine du BTP, en mettant en évidence leur pertinence et leur impact sur l'industrie du BTP. Nous avons également exposé notre projet de conception et de mise en place d'une telle plateforme en utilisant la méthode 2TUP. Tout au long de notre travail, nous avons suivi une approche méthodologique rigoureuse en décrivant le contexte de notre étude, analysant l'existant, en spécifiant les besoins fonctionnels et techniques, en concevant le système et en l'implémentant.

Ce projet a été une opportunité pour nous d'appliquer nos connaissances théoriques en génie logiciel dans un contexte pratique. Nous avons acquis une expérience précieuse en modélisation UML, en développement mobile et en travail d'équipe.

Bien que nous n'ayons pas implémentés toutes les fonctionnalités de l'application dans le cadre de nos travaux de diplôme, nous sommes toutefois contraints de continuer à travailler sur ce projet afin de le mettre à disposition de notre communauté et de contribuer au développement du secteur du BTP. Ce projet nous a permis de réaliser l'importance de la mise en place de plateformes de mise à disposition des ouvriers en ligne, en particulier dans des contextes où il est difficile de trouver rapidement des ouvriers qualifiés. Nous espérons que ce travail pourra contribuer à encourager la mise en place de telles plateformes dans d'autres régions du pays.

Bibliographie & Webographie

- [1] Wikipédia, Domaine du BTP [Construction — Wikipédia \(wikipedia.org\)](#) [consulté le 14 mars 2023].
- [2] Wikipédia, langage de modélisation [Langage de modélisation — Wikipédia \(wikipedia.org\)](#) [Consulté le 14 mars 2023].
- [3] Wikipédia, Processus Unifié [Processus unifié — Wikipédia \(wikipedia.org\)](#) [consulté le 14 mars 2023].
- [4] Historique de mon-ouvrier [BATIMENT | MON-OUVRIER](#) [consulté le 14 mars 2023].
- [5] Historique de Habitatpresto [Trouvez le bon Professionnel pour vos Travaux | Habitatpresto](#) [consulter le 14 mars 2023].

Table des matières

REMERCIEMENTS	II
LISTE DES FIGURES	III
LISTE DES TABLEAUX	IV
ABREVIATIONS & SIGLES	V
SOMMAIRE	VI
INTRODUCTION	1
CHAPITRE 1 : PRESENTATION DE LA PROBLEMATIQUE	2
1.1 DOMAINE	2
1.2 CONTEXTE DU DOMAINE	2
1.3 PROBLEMATIQUE DU SUJET	2
1.4 INTERET DU SUJET	2
1.5 CONCEPT LIE AU SUJET	2
CHAPITRE 2 : APPROCHE METHODOLOGIQUE	4
2.1 LANGAGE DE MODELISATION	4
2.2 PRESENTATION D'UML	4
2.3 LES TYPES DE DIAGRAMMES D'UML	5
2.3.1.1 Les diagrammes structurels ou statiques	5
2.3.1.2 Les diagrammes comportementaux	5
2.3.1.3 Les diagrammes d'interaction ou dynamique	6
2.4 PROCESSUS DE DEVELOPPEMENT	6
2.5 PRESENTATION DU PROCESSUS UNIFIE (UP)	6
2.6 PRESENTATION DU PROCESSUS 2TUP	7
2.7 2TUP ET UML	8
2.7.1 Description d'UML	8
2.7.2 Catalogue des diagrammes d'UML	9
CHAPITRE 3 : ETUDE DE L'EXISTANT	10
3.1 CONSTATATIONS	10
3.2 SOLUTIONS EXISTANTES	10
3.3 PROPOSITION DE SOLUTION	15
3.4 CHOIX DE LA SOLUTION	15
CHAPITRE 4 : ANALYSE	16
4.1 ANALYSE DES BESOINS	16
4.2 SPECIFICATION FONCTIONNELLE	16
4.2.1 Identification des acteurs	16
4.2.2 DIAGRAMME DE CONTEXTE STATIQUE	17
4.2.3 Identification des cas d'utilisation	17
4.2.4 CLASSIFICATION PAR PACKAGE	19
4.2.5 Relation entre cas d'utilisation (Use case)	21
4.2.6 Classification des cas d'utilisation (priorités, risque et itération)	21
4.3 SPECIFICATIONS DETAILLEES	23
4.4 DIAGRAMME DE SEQUENCE	25
4.5 DIAGRAMME D'ACTIVITE	28

CHAPITRE 5 : CONCEPTION	30
5.1 DIAGRAMME DE CAS D'UTILISATION	30
5.1.1 DIAGRAMME DE CAS D'UTILISATION DE GESTION DU REFERENTIEL DU DOMAINE DU BTP	30
5.1.2 DIAGRAMME DE CAS D'UTILISATION DE GESTION DES OFFRES ET RESERVATIONS	31
5.1.3 DIAGRAMME DE CAS D'UTILISATION DE GESTION DES QUALIFICATIONS	31
5.1.4 DIAGRAMME DE CAS D'UTILISATION DE GESTION DES DISCUSSIONS	32
5.1.5 <i>Diagramme de cas d'utilisation de gestion des comptes utilisateurs</i>	32
5.2 DEVELOPPEMENT DU MODELE STATIQUE	33
5.3 DIAGRAMME DE CLASSE DU SYSTEME	34
5.3.1 <i>Diagramme de classe du référentiel du domaine BPT</i>	34
5.3.2 <i>Diagramme de classe de gestion des offres et réservations</i>	34
5.3.3 <i>Diagramme de classe de gestion des qualifications des ouvriers</i>	35
5.3.4 <i>Diagramme de gestion des discussions</i>	35
5.3.5 <i>Diagramme de classe de gestion des comptes utilisateurs</i>	35
5.3.6 <i>Diagramme de classe global du système</i>	36
5.4 CONCEPTION ARCHITECTURALE	37
5.4.1 <i>Architecture logicielle</i>	37
5.5 DIAGRAMME DE COMPOSANT	39
CHAPITRE 6 : IMPLEMENTATION	40
6.1 ENVIRONNEMENT TECHNIQUE DE DEVELOPPEMENT	40
6.1.1 <i>Environnement logiciel de production de la solution</i>	40
6.1.2 <i>Environnement matériel de production de la solution</i>	44
6.2 DIAGRAMME DE DEPLOIEMENT	45
6.3 PRODUCTION DU LOGICIEL	45
6.3.1 <i>Schéma relationnel</i>	45
6.3.2 <i>Modèle physique des données</i>	47
6.3.3 <i>Code source de l'application</i>	49
6.3.4 <i>Extraits des écrans de l'application</i>	53
CONCLUSION	56
BIBLIOGRAPHIE & WEBOGRAPHIE	A

Résumé :

Dans le cadre de notre projet de fin d'études, nous entreprenons le développement d'une plateforme numérique novatrice orientée vers la mise à disposition d'ouvriers dans le secteur du BTP. Notre objectif est de faciliter la réservation et la gestion des ouvriers pour répondre aux besoins spécifiques des entreprises du domaine BTP. Cette plateforme permettra de coordonner efficacement l'offre et la demande des ressources humaines en reliant sur une plateforme les employeurs et les ouvriers qualifiés en vue des prestations de travaux.

La conception de cette plateforme sera guidée par la méthodologie 2TUP. La méthode 2TUP est un processus de développement logiciel dérivé du processus unifié. En suivant cette approche, nous organisons les différentes étapes du cycle de développement en mettant l'accent sur la séparation entre les aspects techniques et les aspects fonctionnels du produit numérique à concevoir.

Nous procéderons à une spécification détaillée des besoins fonctionnels en utilisant le langage de modélisation UML (Unified Modeling Language) pour formaliser les données et les interactions des utilisateurs finaux avec le système à concevoir. Pour le stockage et la gestion des informations, nous ferons usage du Système de Gestion des Bases de Données Relationnelles (SGBDR) MySQL.

Notre mission fondamentale consistera à développer une application mobile, baptisée "OPPUSS.", destinée à la fois aux employeurs et aux ouvriers qualifiés. Cette application sera compatible avec les systèmes d'exploitation Android et iOS, en offrant une gamme complète de fonctionnalités pour la réservation, la gestion des missions et des horaires, ainsi que la communication entre les parties prenantes à l'offre et la demande de prestations des travaux de BTP.

Abstract:

As part of our graduation project focused on the availability of workers in the construction sector, we are embarking on the creation of an innovative platform. Our goal is to facilitate the booking and management of workers to meet the specific needs of companies in the field. This platform will make it possible to effectively coordinate human resources by associating employers with skilled workers for various tasks.

The design of this platform will be guided by the 2TUP methodology, a software development process that takes advantage of the principles of the unified process. By following this approach, we organize the different stages of the development cycle in a coherent way, emphasizing the separation between the technical and functional aspects of the project.

Once this methodology has been established, we will proceed to a detailed specification of the functional requirements using the modeling language UML (Unified Modeling Language) to formalize the data and the interactions. The storage and management of information will be entrusted to the Relational Database Management System (RDBMS) MySQL.

Our central mission will be to develop a dual mobile application, called "OPPUSS.", intended for both employers and skilled workers. This application will be compatible with Android and iOS operating systems, offering a full range of functionalities for booking, management of missions and schedules,